

CS336 作业 1（基础）：从零构建 Transformer 语言模型

版本 26.0.3 / 2026 年春季

CS336 课程组

中文翻译版

翻译说明：本文为原作业说明的完整中文翻译。代码标识符、命令、路径、张量形状、公式编号、特殊 token 和参考文献编号尽量按原文保留。本文只翻译题目与说明，不提供作业答案或实现代码。

目录

作业概述

字节对编码（BPE）分词器

Transformer 语言模型架构

训练 Transformer 语言模型

训练循环

生成文本

实验

参考文献

1 作业概述

在本次作业中，你将从零构建训练一个标准 Transformer 语言模型（LM）所需的全部组件，并实际训练若干模型。

你将实现的内容

字节对编码（BPE）分词器（第 2 节）

Transformer 语言模型（LM）（第 3 节）

交叉熵损失函数与 AdamW 优化器（第 4 节）

训练循环，并支持序列化、保存及加载模型和优化器状态（第 5 节）

你将运行的实验

在 TinyStories 数据集上训练 BPE 分词器。

使用训练好的分词器对数据集进行编码，将其转换为整数 ID 序列。

在 TinyStories 数据集上训练 Transformer 语言模型。

使用训练好的 Transformer 语言模型生成样本，并评估困惑度（perplexity）。

在 OpenWebText 上训练模型，并将取得的困惑度提交到排行榜。

允许使用的内容

我们要求你从零构建每个组件。尤其是，除下列内容外，不得使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义：

`torch.nn.Parameter`

`torch.nn` 中的容器类，例如 `Module`、`ModuleList`、`Sequential` 等¹

`torch.optim.Optimizer` 基类

你可以使用 PyTorch 中的其他定义。若你不确定某个函数或类是否允许使用，可以在 Slack 上询问。拿不准时，请思考使用它是否破坏了本作业“从零实现”的精神。

关于 AI 工具的声明

AI 可以完全自主地解决作业中的许多部分，这会让学生更难深入参与课程材料并真正学会相关内容。

允许使用 AI 工具回答高层概念问题，或查询函数签名、库 API 等底层编程文档。但不允许使用 AI 工具实现本作业的任何部分。这既包括编码智能体（如 Cursor Agents、Codex、Claude Code），也包括 AI 自动补全（如 Cursor Tab、GitHub Copilot）。使用 AI 智能体时，请确保它遵循课程提供的 AGENTS.md 文件；使用聊天机器人时，也应附上所用提示词。

我们强烈建议你在完成作业时关闭 IDE 中的 AI 自动补全功能（例如 Cursor Tab、GitHub Copilot）。非 AI 的普通补全，例如函数名补全，完全没有问题。往届学生反馈，关闭 AI 自动补全后，更容易深入理解课程内容。

完整的 AI 政策见原文所链接的文档。

代码仓库结构

作业代码和本说明文档均位于 GitHub：

github.com/stanford-cs336/assignment1-basics

请使用 `git clone` 克隆仓库。如果有更新，课程组会通知你执行 `git pull` 获取最新版本。

`cs336_basics/*`：你编写代码的位置。这里没有预置代码，你可以完全从零组织实现。

`adapters.py`：这里定义了你的代码必须提供的一组功能接口。对于每项功能（例如缩放点积注意力），只需在相应适配函数（例如 `run_scaled_dot_product_attention`）中调用你的实现。注意：对 `adapters.py` 的修改不应包含实质性逻辑，它只是胶水代码。

¹ 完整容器类列表见 pytorch.org/docs/stable/nn.html#containers。

test_*.py: 包含你必须通过的全部测试（例如 test_scaled_dot_product_attention），这些测试会调用 adapters.py 中定义的接口。不要修改测试文件。

提交方式

运行 make_submission.sh 构建提交用的 zip 文件。如果你有不希望打包的大型数据文件或检查点，请把相应文件加入脚本的排除列表。

你需要向 Gradescope 提交：

writeup.pdf: 回答所有书面问题，请排版后提交。

code.zip: 包含你编写的全部代码。

若要提交排行榜成绩，请向以下仓库发起 PR：

github.com/stanford-cs336/assignment1-basics-leaderboard

详细说明见该排行榜仓库中的 README.md。

数据集获取方式

本作业使用两个经过预处理的数据集：TinyStories [R. Eldan 等, 2023] 和 OpenWebText [A. Gokaslan 等, 2019]。两者都是单个大型纯文本文件。

若你正式选修本课程，请查看计算资源指南中的下载说明。

若你在家自行学习，可使用仓库 README.md 中的命令下载这些文件。

低资源提示：说明

在课程作业说明中，我们会给出在 GPU 较少甚至没有 GPU 的情况下完成作业的建议。例如，缩小数据集或模型规模，或者说明如何在 Mac 集成 GPU 或 CPU 上训练。你会看到类似本段的“低资源提示”框。即使你是能使用课程服务器的在校学生，这些建议也有助于更快迭代并节省时间，建议阅读。

低资源提示：在 Apple Silicon 或 CPU 上完成作业 1

使用课程组参考实现，在一台配备 36 GB 内存的 Apple M4 Max 上，通过 Metal GPU (MPS) 不到 5 分钟即可训练出一个能生成较流畅文本的语言模型；使用 CPU 大约需要 30 分钟。即使你不熟悉这些术语也不必担心：只要笔记本较新、实现正确且高效，你就能训练出一个可以较流畅生成简单儿童故事的小型语言模型。后文会说明使用 CPU 或 MPS 时需要做哪些调整。

2 字节对编码（BPE）分词器

本作业的第一部分将训练并实现一个字节级字节对编码（byte-level byte-pair encoding, BPE）分词器 [R. Sennrich 等, 2016; C. Wang 等, 2019]。具体来说，我们把任意 Unicode 字符串表示成字节序列，并在该字节序列上训练 BPE 分词器。之后，我们会使用该分词器把文本字符串编码成 token（整数序列），供语言建模使用。

2.1 Unicode 标准

Unicode 是一种文本编码标准，它把字符映射到整数码点（code point）。截至 Unicode 17.0（2025 年 9 月发布），该标准在 172 种书写系统中定义了 159,801 个字符。例如，字符 s 的码点是 115，通常写作 U+0073，其中 U+ 是约定前缀，0073 是 115 的十六进制表示；字符“牛”的码点是 29275。

在 Python 中，可以用 `ord()` 把单个 Unicode 字符转换成整数表示，用 `chr()` 把整数码点转换成对应字符的字符串：

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

问题（unicode1）：理解 Unicode（1 分）

(a) `chr(0)` 返回哪个 Unicode 字符？

提交内容：一句话回答。

(b) 该字符的字符串表示（`__repr__()`）与直接打印出来的表示有何不同？

提交内容：一句话回答。

(c) 当该字符出现在文本中时会发生什么？可在 Python 解释器中尝试以下代码，观察结果是否符合预期：

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

提交内容：一句话回答。

2.2 Unicode 编码

Unicode 标准定义了字符到码点（整数）的映射，但直接在 Unicode 码点上训练分词器并不现实，因为词表会大得难以接受（约 15 万项），而且非常稀疏（许多字符极少出现）。因此，我们会使用一种 Unicode 编码，把 Unicode 字符转换为字节序列。Unicode 标准本身定义了 UTF-8、UTF-16 和 UTF-32 三种编码，其中 UTF-8 是互联网的主流编码，超过 98% 的网页使用它。

在 Python 中，可以用 `encode()` 把 Unicode 字符串编码为 UTF-8。若要访问 Python bytes 对象底层的字节值，可以对它进行迭代，例如调用 `list()`。最后，可使用 `decode()` 将 UTF-8 字节串解码回 Unicode 字符串。

```
>>> test_string = "hello! こんにちは!"
>>> utf8_encoded = test_string.encode("utf-8")
>>> print(utf8_encoded)
b'hello! \xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf!'
>>> print(type(utf8_encoded))
<class 'bytes'>
>>> # 获取编码后字符串的字节值 (0 到 255 的整数)
>>> list(utf8_encoded)
[104, 101, 108, 108, 111, 33, 32, 227, 129, 147, 227, 130, 147, 227, 129, 171, 227, 129,
161, 227, 129, 175, 33]
>>> # 一个字节不一定对应一个 Unicode 字符!
>>> print(len(test_string))
13
>>> print(len(utf8_encoded))
```

```
23
>>> print(utf8_encoded.decode("utf-8"))
hello! こんにちは!
```

把 Unicode 码点转换成字节序列（例如使用 UTF-8），本质上是把一串码点（21 位整数，其中有 159,801 个合法值）转换成一串取值范围为 0 到 255 的字节。长度为 256 的字节词表更容易处理。使用字节级分词时，我们也不必担心未登录词（out-of-vocabulary），因为任何输入文本都能表示为 0 到 255 的整数序列。

问题（unicode2）：Unicode 编码（3 分）

(a) 与 UTF-16 或 UTF-32 相比，为什么更适合在 UTF-8 编码后的字节上训练分词器？比较不同输入字符串在这些编码下的输出可能会有帮助。

提交内容：一到两句话。

(b) 考虑下面这个错误的函数。它本来想把 UTF-8 字节串解码成 Unicode 字符串。为什么它是错的？请给出一个会产生错误结果的输入字节串。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

提交内容：给出一个使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的字节串，并用一句话解释该函数为什么错误。

(c) 给出一个无法解码为任何 Unicode 字符的双字节序列。

提交内容：一个例子及一句解释。

2.3 子词分词

字节级分词能够避免词级分词器的未登录词问题，但把文本拆成字节会产生非常长的输入序列。一个包含 10 个单词的句子，在词级语言模型中可能只有 10 个 token，但在字符级模型中可能有 50 个甚至更多 token，具体取决于单词长度。处理更长的序列会增加模型每一步的计算量；而且，由于输入序列变长，字节序列上的语言建模还会产生更长程的数据依赖，因此难度更大。

子词分词位于词级分词与字节级分词之间。字节级分词器的初始词表有 256 项（字节值为 0 到 255）。子词分词器用更大的词表换取对输入字节序列更好的压缩。例如，如果字节序列 `b'the'` 在原始训练文本中频繁出现，就可以把它加入词表，从而将原本 3 个 token 的序列压缩为 1 个 token。

那么，应该如何选择加入词表的子词单元？R. Sennrich 等 [3] 提出使用字节对编码（BPE；P. Gage [5]）。BPE 是一种压缩算法，它反复把出现频率最高的一对字节替换（“合并”）为一个新的、此前未使用的索引。这个算法会向词表中加入能最大化输入序列压缩率的子词 token；如果某个单词在输入文本中出现足够多，它可能最终会被表示为一个单独的子词单元。

通过 BPE 构建词表的子词分词器通常称为 BPE 分词器。本作业将实现一个**字节级 BPE 分词器**：词表项要么是单个字节，要么是若干字节合并后的序列。这既保留了字节级分词不产生未登录词的优势，也能把输入序列控制在合理长度。构建 BPE 分词器词表的过程称为“训练”BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器的训练过程主要有三个步骤。

初始化词表

分词器词表是从字节串 token 到整数 ID 的一一映射。由于训练的是字节级 BPE 分词器，初始词表就是全部单字节值。字节共有 256 种可能取值，因此初始词表大小为 256。

预分词

有了词表之后，原则上可以直接统计整份文本中相邻字节对的出现次数，并从最频繁的字节对开始合并。但这样计算代价很高，因为每进行一次合并，都需要完整扫描一遍语料。此外，直接跨整份语料合并字节，可能生成仅标点不同的 token，例如 `dog!` 和 `dog.`。它们会获得完全不同的 token ID，尽管二者语义可能高度相似，因为仅仅末尾标点不同。

为避免这一问题，我们先对语料进行**预分词**。可以把它理解为对语料执行一次粗粒度分词，以便高效统计字符对的出现频率。例如，单词 `text` 可能作为预 token 出现了 10 次。当统计 `t` 与 `e` 相邻出现的次数时，我们知道 `text` 内部有 `t`、`e` 相邻，于是可以一次把计数增加 10，而不必逐字遍历整份语料。由于训练的是字节级 BPE 模型，每个预 token 都表示成 UTF-8 字节序列。

R. Sennrich 等 [3] 的原始 BPE 实现仅按空白切分进行预分词，即 `s.split(" ")`。这种做法至今仍能在基于 SentencePiece 的分词器中看到，例如 Llama 1 和 Llama 2 的分词器。

大多数现代分词器采用基于正则表达式的预分词器，这一做法源自 GPT-2 [A. Radford 等, 6]。本作业采用原正则表达式的一种更整洁的写法，来自：

github.com/openai/tiktoken/pull/234/files

```
>>> PAT = r"\"(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[\s\p{L}\p{N}]+|\s+(?!\S)|\s+\""
```

可以交互式地用该预分词器切分一些文本，以理解它的行为：

```
>>> # 需要安装 'regex' 包
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', 'll', ' pre', '-', 'tokenize']
```

不过在实际代码中，应使用 `re.finditer`，这样在构建“预 token 到计数”的映射时，不需要先把全部预分词结果存入内存。

计算 BPE 合并

现在，输入文本已被转成预 token，每个预 token 又被表示为 UTF-8 字节序列，因此可以开始计算 BPE 合并，也就是训练 BPE 分词器。

从高层看，BPE 算法会反复统计每一对相邻字节，并找出频率最高的一对，例如 (A, B)。随后，把这对字节的每一次出现都合并，即用新 token `AB` 替换。新合并得到的 token 会加入词表。因此，BPE 训练后的最终词表大小等于初始词表大小（本作业中为 256）加上训练过程中执行的 BPE 合并次数。

为了提高训练效率，我们不考虑跨越预 token 边界的字节对。² 当多个字节对频率相同时，应采用确定性的规则打破平局：选择字典序更大的字节对。例如，若 (A, B)、(A, C)、(B, ZZ) 和 (BA, A) 的频率并列最高，则合并 (BA, A)：

```
>>> max([("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A")])
('BA', 'A')
```

特殊 token

一些字符串，例如 `<|endoftext|>`，常用于编码元数据，例如文档边界。在编码文本时，我们通常希望把某些字符串视为“特殊 token”，永远不把它们拆成多个 token，而是始终作为一个整体保留。例如，序列结束字符串 `<|endoftext|>` 应始终对应一个整数 ID，这样语言模型生成到它时就知道应该停止。特殊 token 必须加入词表，以获得固定的 token ID。

R. Sennrich 等 [3] 的算法 1 给出了一个效率较低的 BPE 分词器训练实现，基本上就是按上述步骤执行。作为第一个练习，你可以先实现并测试这个朴素版本，以检验自己对算法的理解。

示例 (bpe_example)：BPE 训练示例

下面是改写自 R. Sennrich 等 [3] 的示意例子。假设语料由以下文本组成：

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

词表中还有一个特殊 token `<|endoftext|>`。

词表

我们使用特殊 token `<|endoftext|>` 和全部 256 个字节值初始化词表。

预分词

为简化示例并把重点放在合并过程上，这里假设预分词只按空白切分。预分词并计数后，得到频率表：

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

可以方便地将其表示为 `dict[tuple[bytes, ...], int]`，例如 `{(l,o,w): 5, ...}`。注意，在 Python 中，即使是单个字节也要用 `bytes` 对象表示；Python 没有单独的 `byte` 类型来表示一个字节，就像它也没有 `char` 类型来表示一个字符。

合并过程

首先查看每一对相邻字节，并把它们所在单词的频率相加，得到：

```
{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}
```

字节对 ('e', 's') 与 ('s', 't') 频率相同，因此选择字典序更大的 ('s', 't')。合并后，预 token 表示变为：

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}
```

² 原始 BPE 方案 [R. Sennrich 等, 3] 规定加入一个词尾 token。本作业训练字节级 BPE 模型时不加入词尾 token，因为全部字节（包括空白和标点）都已包含在模型词表中。由于空格和标点被显式表示，学到的 BPE 合并会自然反映词边界。

第二轮中, (e, st) 是最常见的字节对, 计数为 9, 因此合并后得到:

```
{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}
```

继续执行, 最终的合并顺序为:

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e',  
'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']
```

若只执行前 6 次合并, 则合并列表为:

```
['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']
```

此时词表元素为:

```
[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]
```

使用该词表和合并列表, 单词 newest 会被分成 [ne, west]。

2.5 BPE 分词器训练实验

接下来在 TinyStories 数据集上训练字节级 BPE 分词器。数据集的查找和下载说明见第 1 节。开始之前, 建议先浏览 TinyStories, 了解数据内容。

并行化预分词

你会发现, 预分词是主要性能瓶颈之一。可以使用 Python 内置的 multiprocessing 库对预分词并行化。具体而言, 在并行实现中, 建议把语料切成若干块, 并确保每个分块边界都位于某个特殊 token 的开头。你可以原样使用以下链接中的起始代码来确定分块边界, 再把各块分发给不同进程:

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

这种分块方式始终有效, 因为我们本就不希望在文档边界之间执行合并。对本作业来说, 可以始终按这种方式切分, 不必考虑超大语料中完全没有 <|endoftext|> 的边缘情况。

预分词前移除特殊 token

使用正则表达式执行预分词 (即使用 re.finditer) 前, 应先从语料中移除全部特殊 token; 若采用并行实现, 则对每个分块执行相同操作。必须按特殊 token 切分文本, 确保不会跨越它们界定的边界进行合并。

例如, 若语料或分块是 [Doc 1]<|endoftext|>[Doc 2], 应以 <|endoftext|> 为分隔符切分, 并分别对 [Doc 1] 与 [Doc 2] 进行预分词, 避免跨文档边界合并。换言之, 特殊 token 在训练时定义了不可跨越的硬分段边界, 但特殊 token 本身不参与合并频率统计。

可以把 "|" .join(special_tokens) 作为分隔符传给 re.split; 由于特殊 token 中可能包含 |, 需谨慎使用 re.escape。测试 test_train_bpe_special_tokens 会检查这一点。

优化合并步骤

前面示例中的朴素 BPE 训练实现很慢, 因为每次合并都要遍历所有字节对, 找出频率最高的一对。但每次合并后, 实际会发生计数变化的, 仅仅是与被合并字节对重叠的那些字节对。因此, 可以为所有字节对的计数建立索引, 并在合并后增量更新相关计数, 而不是每轮重新遍历并统计全部字节对。使用这种缓存策略可以获得显著加速。不过, Python 中的 BPE 合并步骤本身无法并行化。

低资源提示：性能分析

使用 cProfile 或 py-spy 等性能分析工具找出实现中的瓶颈，并优先优化它们。

低资源提示：“缩小规模”

不要一上来就在完整 TinyStories 训练集上训练分词器。建议先使用较小的数据子集作为“调试数据集”。例如，可以先在 TinyStories 验证集上训练；验证集只有约 2.2 万篇文档，而训练集有约 212 万篇。这体现了一个通用策略：尽可能缩小数据集、模型和其他规模，以加快开发。调试数据集或超参数配置的大小需要认真选择：既要足够大，使其具有与完整配置相同的瓶颈，从而保证你的优化可以迁移；又不能大到每次运行都耗时过长。

问题 (train_bpe)：训练 BPE 分词器 (15 分)

提交内容：编写一个函数，给定输入文本文件路径，训练一个字节级 BPE 分词器。你的 BPE 训练函数至少应支持以下输入参数。

输入

`input_path: str`: 包含 BPE 分词器训练数据的文本文件路径。

`vocab_size: int`: 正整数，定义最终词表的最大大小。该大小包括初始字节词表、合并产生的词表项以及所有特殊 token。

`special_tokens: list[str]`: 要加入词表的一组字符串。训练过程中，将它们视为不可跨越的硬边界，禁止跨越其范围进行合并；但计算合并统计时不应计入它们。

函数应返回训练得到的词表和合并列表。

输出

`vocab: dict[int, bytes]`: 分词器词表，从整数 token ID 映射到 token 对应的字节串。

`merges: list[tuple[bytes, bytes]]`: 训练得到的 BPE 合并列表。每个元素是一个字节串元组 (`<token1>`, `<token2>`)，表示 `<token1>` 与 `<token2>` 被合并。列表顺序必须与合并创建顺序一致。

若要用课程提供的测试验证函数，需先实现测试适配器 `[adapters.run_train_bpe]`，再运行：

```
uv run pytest tests/test_train_bpe.py
```

你的实现应通过全部测试。

可选地，你可以使用系统语言实现训练方法的关键部分，例如 C++（可考虑 `cppyy` 或 `nanobind`）或 Rust（使用 `PyO3`），但这可能需要投入大量时间。采用这种方式时，需要注意哪些操作会复制 Python 内存、哪些可以直接读取，并提供构建说明，或确保只通过 `pyproject.toml` 即可完成构建。

还要注意，大多数正则表达式引擎对 GPT-2 正则支持不佳，而且运行速度过慢。课程组验证过 `Oniguruma` 性能尚可且支持负向先行断言，但 Python 的 `regex` 包甚至可能更快。

问题 (train_bpe_tinystories)：在 TinyStories 上训练 BPE (2 分)

(a) 在 TinyStories 数据集上训练字节级 BPE 分词器，最大词表大小设为 10,000。务必把 TinyStories 的 `<|endoftext|>` 特殊 token 加入词表。将训练得到的词表和合并列表序列化到磁盘，供后续检查。训练耗费了多少时间和内存？词表中最长的 token 是什么？这个结果是否合理？

资源限制：不使用 GPU 时不超过 30 分钟，内存不超过 30 GB。

提示：若在预分词阶段使用多进程，并利用以下两点，BPE 训练应能控制在 2 分钟以内：

(a) 数据文件中的 `<|endoftext|>` 用于分隔文档；

(b) `<|endoftext|>` 在应用 BPE 合并前作为特殊情况处理。

提交内容：一到两句话。

(b) 对代码进行性能分析。分词器训练过程中哪一部分耗时最多？

提交内容：一到两句话。

接下来，尝试在 OpenWebText 数据集上训练字节级 BPE 分词器。和之前一样，建议先浏览数据集，以理解其中内容。

问题 (train_bpe_expts_owt)：在 OpenWebText 上训练 BPE (2 分)

(a) 在 OpenWebText 数据集上训练字节级 BPE 分词器，最大词表大小为 32,000。把生成的词表和合并列表序列化到磁盘，供进一步检查。词表中最长的 token 是什么？这个结果是否合理？

资源限制：不使用 GPU 时不超过 12 小时，内存不超过 100 GB。

提交内容：一到两句话。

(b) 比较在 TinyStories 与 OpenWebText 上训练得到的分词器，有哪些相同与不同之处？

提交内容：一到两句话。

2.6 BPE 分词器：编码与解码

上一部分中，我们实现了一个函数，在输入文本上训练 BPE 分词器，得到词表和 BPE 合并列表。现在要实现一个 BPE 分词器，它可以加载给定的词表和合并列表，并在文本与 token ID 之间进行编码和解码。

2.6.1 编码文本

BPE 编码文本的过程与训练 BPE 词表的过程相似，主要包含以下步骤。

步骤 1：预分词。与 BPE 训练时一样，先对序列进行预分词，并把每个预 token 表示成 UTF-8 字节序列。随后只在单个预 token 内部合并这些字节，各预 token 独立处理，不跨预 token 边界合并。

步骤 2：应用合并。按 BPE 训练时的创建顺序，依次把训练得到的词表元素合并规则应用到各预 token。

示例 (bpe_encoding) : BPE 编码示例

假设输入字符串为 'the cat ate', 词表为:

```
{0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't',  
6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}
```

学到的合并规则为:

```
[(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a', b't')]
```

首先, 预分词器把字符串切成:

```
['the', ' cat', ' ate']
```

然后分别对每个预 token 应用 BPE 合并。

第一个预 token 'the' 最初表示为 [b't', b'h', b'e']。查看合并列表, 第一条可应用的规则是 (b't', b'h'), 应用后变为 [b'th', b'e']。随后重新检查合并列表, 下一条可应用规则为 (b'th', b'e'), 于是变为 [b'the']。此后没有更多规则可应用, 因为整个预 token 已合并为一个 token。其对应整数序列是 [9]。

对剩余预 token 重复该过程。' cat' 应用合并后表示为 [b' c', b'a', b't'], 对应整数序列 [7, 1, 5]。最后一个预 token ' ate' 表示为 [b' at', b'e'], 对应整数序列 [10, 3]。因此, 完整输入最终被编码为:

```
[9, 7, 1, 5, 10, 3]
```

特殊 token

编码文本时, 分词器应能正确处理用户定义的特殊 token; 这些 token 在构造分词器时提供。

内存方面的考虑

假设要对一个无法完整放入内存的大型文本文件进行分词。为了高效处理该文件或其他数据流, 需要把它拆成大小可控的块, 逐块处理, 使内存复杂度保持为常数, 而不是随文本长度线性增长。同时必须确保 token 不会跨越分块边界, 否则结果会不同于把整段文本一次性加载到内存后进行分词的朴素方法。

2.6.2 解码文本

要把整数 token ID 序列解码回原始文本, 只需在词表中查找每个 ID 对应的字节序列, 把所有字节拼接起来, 再解码为 Unicode 字符串。

注意, 输入 ID 不保证一定组合成合法的 Unicode 字符串, 因为用户可以传入任意整数 ID 序列。如果得到的字节串不是合法 Unicode, 应使用 Unicode 官方替换字符 U+FFFD 替换格式错误的字节。³

bytes.decode 的 errors 参数控制 Unicode 解码错误的处理方式, 设置 errors='replace' 会自动用替换字符代替格式错误的数据。

问题 (tokenizer) : 实现分词器 (15 分)

提交内容: 实现一个 Tokenizer 类。给定词表与合并列表, 它应能把文本编码为整数 ID, 也能把整数 ID 解码为文本。分词器还应支持用户提供的特殊 token; 若这些 token 尚未存在于词表中, 就将它们追加到词表。

³ 关于 Unicode 替换字符的更多信息, 见 [en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character)。

建议使用以下接口：

```
def __init__(self, vocab, merges, special_tokens=None)
```

根据给定词表、合并列表，以及可选的特殊 token 列表构造分词器。参数：

vocab: dict[int, bytes]

merges: List[tuple[bytes, bytes]]

special_tokens: list[str] | None = None

```
def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)
```

类方法：从序列化后的词表和合并列表文件中构造并返回 Tokenizer。文件格式应与你的 BPE 训练代码输出格式一致；还可传入特殊 token 列表。额外参数：

vocab_filepath: str

merges_filepath: str

special_tokens: list[str] | None = None

```
def encode(self, text: str) -> List[int]
```

把输入文本编码为 token ID 序列。

```
def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]
```

给定一个字符串可迭代对象（例如 Python 文件句柄），返回一个惰性地产生 token ID 的生成器。这是对无法一次装入内存的大型文件进行内存高效分词所必需的。

```
def decode(self, ids: List[int]) -> str
```

把 token ID 序列解码为文本。

若要用提供的测试验证 Tokenizer，需先实现测试适配器 [adapters.get_tokenizer]，再运行：

```
uv run pytest tests/test_tokenizer.py
```

你的实现应通过全部测试。

2.7 实验

问题 (tokenizer_experiments)：分词器实验（4 分）

(a) 分别从 TinyStories 和 OpenWebText 抽取 10 篇文档。使用此前训练的 TinyStories 分词器（词表大小 10K）与 OpenWebText 分词器（词表大小 32K），把相应样本文档编码为整数 ID。每个分词器的压缩率（字节数/token）是多少？

提交内容：一到两句话。

(b) 如果用 TinyStories 分词器对 OpenWebText 样本进行分词，会发生什么？比较压缩率，并/或定性描述结果。

提交内容：一到两句话。

(c) 估计分词器吞吐量，例如每秒处理多少字节。若要对 825 GB 文本的 Pile 数据集进行分词，大约需要多久？

提交内容：一到两句话。

(d) 使用 TinyStories 与 OpenWebText 分词器，分别把对应训练集和开发集编码为整数 token ID 序列。后续将用这些数据训练语言模型。建议把 token ID 序列化为数据类型为 uint16 的 NumPy 数组。为什么 uint16 是合适的选择？

提交内容： 一到两句话。

3 Transformer 语言模型架构

语言模型的输入是一批整数 token ID 序列，即形状为 $(\text{batch_size}, \text{sequence_length})$ 的 `torch.Tensor`；输出则是词表上的一批归一化概率分布，即形状为 $(\text{batch_size}, \text{sequence_length}, \text{vocab_size})$ 的 PyTorch 张量。对于输入中的每个 token 位置，模型给出的预测分布都对应下一个 token。

训练语言模型时，我们使用这些“下一个 token”预测，计算真实下一个 token 与预测下一个 token 之间的交叉熵损失。推理生成文本时，我们取最后一个时间步，也就是序列最后一个位置上的下一个 token 概率分布，并据此生成序列中的下一个 token，例如选择概率最大的 token，或从分布中采样。然后把生成出的 token 追加到输入序列中，并重复该过程。

本部分将从零构建这个 Transformer 语言模型。我们先从高层描述模型，再逐步讲解每个组件。

3.1 Transformer 语言模型

给定 token ID 序列，Transformer 语言模型先使用输入嵌入把 token ID 转换成稠密向量，再让嵌入后的 token 依次通过 `num_layers` 个 Transformer 块，最后应用一个可学习的线性投影，也称“输出嵌入”或“LM head”，得到预测下一个 token 的 logits。图 1 给出了示意结构。

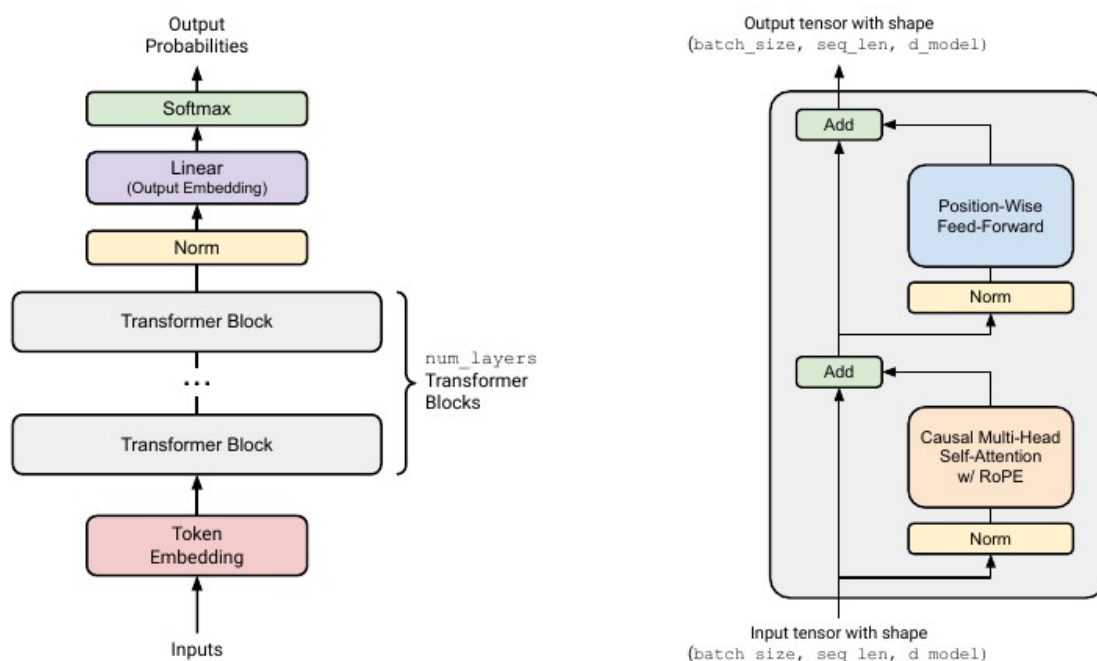


图 1: Transformer 语言模型概览；图 2: Pre-Norm Transformer 块。图中英文标签保留原文。

Token 嵌入

在第一步中，Transformer 把一批 token ID 序列嵌入成一串向量，这些向量包含 token 身份信息，对应图 1 中的红色模块。

更具体地说，token 嵌入层接收形状为 $(batch_size, sequence_length)$ 的整数张量，输出形状为 $(batch_size, sequence_length, d_model)$ 的向量序列。

Pre-Norm Transformer 块

完成嵌入后，激活值会经过若干结构相同的神经网络层。标准的 decoder-only Transformer 语言模型包含 num_layers 个相同的层，通常称为 Transformer “块”。每个 Transformer 块接收形状为 $(batch_size, sequence_length, d_model)$ 的输入，并返回同样形状的输出。每个块通过自注意力聚合序列中的信息，再通过前馈层对信息进行非线性变换。

经过 num_layers 个 Transformer 块后，我们会把最终激活值转换成词表上的概率分布。

本作业实现的是“pre-norm”Transformer 块，详见第 3.4 节。该架构还要求在最后一个 Transformer 块之后额外使用一次层归一化，以确保输出尺度合适。

完成该归一化后，使用标准的可学习线性变换，把 Transformer 块输出转换成预测下一个 token 的 logits，可参考 A. Radford 等 [7] 的公式 2。

3.2 说明：批处理、Einsum 与高效计算

在 Transformer 的不同位置，我们都会对许多“类似批次”的输入重复执行相同计算。例如：

批元素素：对 batch 中的每个样本执行相同的 Transformer 前向计算。

序列长度：RMSNorm、前馈网络等“逐位置”运算，会对序列的每个位置执行相同计算。

注意力头：多头注意力会把注意力运算并行地应用到所有注意力头上。

为了充分利用 GPU，同时让代码易读易懂，我们需要一种便于表达这些运算的方式。许多 PyTorch 运算允许在张量最前面添加额外的“类似批次”的维度，并能高效地在这些维度上重复或广播运算。

例如，假设进行逐位置的批处理运算。数据张量 D 的形状为 $(batch_size, sequence_length, d_model)$ ，矩阵 A 的形状为 (d_model, d_model) ，我们希望执行批量向量-矩阵乘法。在这种情况下， $D @ A$ 会执行批量矩阵乘法，PyTorch 会把 $(batch_size, sequence_length)$ 当作批次维度。这是一个高效的底层操作。

因此，最好假设函数可能接收额外的批次型维度，并始终把这些维度放在 PyTorch 张量形状的最前面。为了把张量整理成合适形状，可能需要多次使用 `view`、`reshape` 和 `transpose`。这种写法比较麻烦，也很容易让人看不清代码在做什么、各张量形状是什么。

更符合人体工学的选择，是在 `torch.einsum` 中使用 `einsum` 记号，或者使用与框架无关的 `einops`、`einx` 库。两个关键操作是：

einsum：对输入张量的任意维度执行张量收缩。

rearrange：对任意维度进行重排、拼接和拆分。

事实上，机器学习中的几乎所有运算，都可以看成维度整理、张量收缩，以及偶尔插入一个通常为逐元素的非线性函数。因此，使用 einsum 记号可以让大量代码更具可读性和灵活性。

课程强烈建议学习并使用 einsum 记号。此前没有接触过 einsum 的学生应先使用 einops；已经熟悉 einops 的学生可以学习更通用的 einx。⁴ 课程环境已经安装这两个包。

下面给出一些 einsum 用法示例。这些内容只是 einops 文档的补充，请先阅读正式文档。

示例 (einstein_example1)：用 einops.einsum 执行批量矩阵乘法

```
import torch
from einops import rearrange, einsum

## 基础实现
Y = D @ A.T
# 很难直接看出输入、输出形状及其含义。
# D 和 A 可以有哪些形状？某些形状是否会产生意外行为？

## Einsum 具有自文档化特性，而且更稳健
#           D           A       ->       Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## 或者使用批处理版本：D 可以有任意数量的前导维度，但 A 的形状受到约束
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

示例 (einstein_example2)：使用 einops.rearrange 完成广播运算

假设有一批图像，希望根据某个缩放因子，为每张图像生成 10 个不同亮度的变暗版本：

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## 先重塑，再相乘
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## 或者一步完成
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

示例 (einstein_example3)：使用 einops.rearrange 混合像素

假设一批图像用形状为 (batch, height, width, channel) 的张量表示。我们希望在整张图像的所有像素之间执行一次线性变换，但每个通道应独立处理。线性变换矩阵 B 的形状为 (height * width, height * width)。

```
channels_last = torch.randn(64, 32, 32, 3) # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## 重排图像张量，以便在全部像素上进行混合
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
```

⁴ einops 已获得广泛支持，而 einx 尚未经过同等程度的实战检验。如果遇到 einx 的限制或错误，可以退回 einops 配合普通 PyTorch。

```

)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)

```

使用 `einops` 可以写成：

```

height = width = 32

## rearrange 取代繁琐的 torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)

```

如果你想写得更激进一些，也可以用 `einx.dot` 一步完成；它相当于 `einops.einsum` 的 `einx` 版本：

```

height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)

```

第一种普通 PyTorch 实现当然可以通过在前后加注释，说明输入输出形状来改进，但这种方式笨重且容易出错。使用 `einsum` 记号时，“文档就是实现”。

`Einsum` 记号既能处理任意输入批次维度，又具备自文档化优势：代码中各输入输出张量的关键形状一目了然。对其余张量，还可以考虑使用 `Tensor` 类型提示，例如 `jaxtyping` 库，尽管它并不只适用于 JAX。

作业 2 会进一步讨论 `einsum` 对性能的影响；目前只需知道，它们几乎总比替代写法更好。

3.2.1 数学记号与内存顺序

许多机器学习论文使用行向量记号，这与 NumPy 和 PyTorch 默认采用的行优先（row-major）内存顺序配合得很好。使用行向量时，线性变换写作：

$$y = x W^T \quad (1)$$

其中，按行优先存储的 $W \in \mathbb{R}^{(d_{\text{out}} \times d_{\text{in}})}$ ，行向量 $x \in \mathbb{R}^{(1 \times d_{\text{in}})}$ 。这样可以通过扩展 x 的最外层维度来批处理输入，即把向量输入 x 替换为矩阵输入 $X \in \mathbb{R}^{(\text{batch} \times d_{\text{in}})}$ 。

在线性代数中，更常见的是使用列向量，此时线性变换写作：

$$y = W x \quad (2)$$

其中 $W \in \mathbb{R}^{(d_{\text{out}} \times d_{\text{in}})}$ 仍按行优先存储，列向量 $x \in \mathbb{R}^{(d_{\text{in}})}$ 。如果在这种记号下批处理输入，`batch` 维必须放到 x 的最后，因此需要把 x 替换成 $\tilde{x} \in \mathbb{R}^{(d_{\text{in}} \times \text{batch})}$ 。

本作业的数学记号大多使用列向量，因为数学文献通常如此。需要注意：PyTorch 使用行优先内存顺序，因此若使用普通矩阵乘法记号实现，必须像公式 1 的行向量约定那样对矩阵做转置。如果用 einsum 执行线性代数运算，只要正确标记各轴，这个问题通常不会出现。

顺带一提，Matlab、Julia 和 Fortran 等语言或线性代数包使用列优先内存顺序，因此 batch 维通常放在最后；Python 及相关包则采用 C 标准的行优先顺序。

3.3 基础构件：线性层与嵌入模块

3.3.1 参数初始化

要有效训练神经网络，通常必须谨慎初始化模型参数。糟糕的初始化会导致梯度消失、梯度爆炸等问题。Pre-norm Transformer 对初始化异常稳健，但初始化仍会显著影响训练速度和收敛。本作业已经很长，因此把细节留到作业 3；这里给出一组适用于大多数情况的近似初始化方案：

线性层权重：从 $N(\mu = 0, \sigma^2 = 2 / (d_{in} + d_{out}))$ 中采样，并截断到 $[-3\sigma, 3\sigma]$ 。

嵌入：从 $N(\mu = 0, \sigma^2 = 1)$ 中采样，并截断到 $[-3, 3]$ 。

RMSNorm：初始化为全 1 向量。

应使用 `torch.nn.init.trunc_normal_` 初始化截断正态分布权重。

3.3.2 线性模块

线性层是 Transformer 和一般神经网络的基础构件。首先，实现一个继承自 `torch.nn.Module` 的自定义 `Linear` 类，执行线性变换：

$$y = W x \quad (3)$$

按照大多数现代大语言模型的做法，这里不包含偏置项。

问题（linear）：实现线性模块（1 分）

提交内容：实现一个继承自 `torch.nn.Module`、执行线性变换的 `Linear` 类。除去不提供偏置参数外，其接口应与 PyTorch 内置的 `nn.Linear` 一致。建议接口如下：

```
def __init__(self, in_features, out_features, device=None, dtype=None)
```

构造线性变换模块。参数：

`in_features: int`：输入的最后一维大小。

`out_features: int`：输出的最后一维大小。

`device: torch.device | None = None`：参数存储设备。

`dtype: torch.dtype | None = None`：参数数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

对输入应用线性变换。

确保做到：

继承 `nn.Module`。

调用父类构造函数。

以 `W` 而不是 `WT` 的形式构造并存储参数，并把它放入 `nn.Parameter`。

不使用 `nn.Linear` 或 `nn.functional.linear`。

参数初始化使用前述设置，并用 `torch.nn.init.trunc_normal_` 初始化权重。

测试时，实现适配器 `[adapters.run_linear]`。适配器应把给定权重加载到你的 `Linear` 模块中，可以使用 `Module.load_state_dict`。然后运行：

```
uv run pytest -k test_linear
```

3.3.3 嵌入模块

如前所述，Transformer 的第一层是嵌入层，它把整数 token ID 映射到 `d_model` 维向量空间。我们将实现一个继承自 `torch.nn.Module` 的自定义 `Embedding` 类，因此不能使用 `nn.Embedding`。

`forward` 方法应使用形状为 `(batch_size, sequence_length)` 的 `torch.LongTensor` token ID，对形状为 `(vocab_size, d_model)` 的嵌入矩阵执行索引，选出每个 token ID 对应的嵌入向量。

问题（embedding）：实现嵌入模块（1 分）

提交内容：实现继承自 `torch.nn.Module` 的 `Embedding` 类，完成嵌入查找。接口应遵循 PyTorch 内置 `nn.Embedding`。建议如下：

```
def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)
```

构造嵌入模块。参数：

`num_embeddings: int`：词表大小。

`embedding_dim: int`：嵌入向量维度，即 `d_model`。

`device: torch.device | None = None`：参数存储设备。

`dtype: torch.dtype | None = None`：参数数据类型。

```
def forward(self, token_ids: torch.Tensor) -> torch.Tensor
```

查找给定 token ID 对应的嵌入向量。

确保做到：

继承 `nn.Module`。

调用父类构造函数。

把嵌入矩阵初始化为 `nn.Parameter`。

存储嵌入矩阵时，使 `d_model` 位于最后一维。

不使用 `nn.Embedding` 或 `nn.functional.embedding`。

同样使用前述初始化方案，并用 `torch.nn.init.trunc_normal_` 初始化权重。

测试时，实现适配器 [adapters.run_embedding]，然后运行：

```
uv run pytest -k test_embedding
```

3.4 Pre-Norm Transformer 块

每个 Transformer 块有两个子层：多头自注意力机制和逐位置前馈网络，见 A. Vaswani 等 [2017] 第 3.1 节。

原始 Transformer 论文在两个子层外分别使用残差连接，然后再执行层归一化。由于层归一化作用在子层输出上，这种结构通常称为 “post-norm” Transformer。

然而，多项研究发现，把层归一化从各子层输出移动到各子层输入，并在最后一个 Transformer 块之后额外添加一次层归一化，可以提高 Transformer 的训练稳定性 [T. Q. Nguyen 等, 2019; R. Xiong 等, 2020]。图 2 展示了这种 “pre-norm” Transformer 块。每个子层的输出再通过残差连接加回子层输入，见 A. Vaswani 等 [8] 第 5.4 节。

一种直觉解释是：pre-norm 在输入嵌入与 Transformer 最终输出之间提供了一条没有被归一化打断的干净 “残差流”，据称这有助于梯度传播。如今 pre-norm Transformer 已成为语言模型中的标准结构，例如 GPT-3、LLaMA、PaLM 等，因此本作业实现这一变体。下面将依次介绍并实现 pre-norm Transformer 块的各组件。

3.4.1 均方根层归一化

A. Vaswani 等 [8] 的原始 Transformer 使用层归一化 [J. L. Ba 等, 2016] 对激活值进行归一化。按照 H. Touvron 等 [12] 的做法，本作业使用均方根层归一化，即 RMSNorm [B. Zhang 等, 13, 公式 4]。

给定激活向量 $a \in \mathbb{R}^{(d_{\text{model}})}$ ，RMSNorm 按如下方式重新缩放每个激活 a_i ：

$$\text{RMSNorm}(a_i) = a_i / \text{RMS}(a) \cdot g_i \quad (4)$$

其中：

$$\text{RMS}(a) = \sqrt{(1 / d_{\text{model}}) \cdot \sum_{i=1}^{d_{\text{model}}} a_i^2 + \epsilon}$$

g_i 是可学习的 “增益” 参数，总共有 d_{model} 个； ϵ 是用于数值稳定性的超参数，通常固定为 $1e-5$ 。

在对输入求平方前，应把输入上转换为 torch.float32，以防溢出。整体 forward 方法应类似：

```
in_dtype = x.dtype
x = x.to(torch.float32)
# 在这里执行 RMSNorm
...
result = ...
# 用原始数据类型返回结果
return result.to(in_dtype)
```

问题 (rmsnorm)：均方根层归一化 (1 分)

提交内容： 把 RMSNorm 实现为 torch.nn.Module。建议接口：

```
def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)
```

参数：

d_model: int: 模型隐藏维度。

eps: float = 1e-5: 用于数值稳定性的 epsilon。

device: torch.device | None = None: 参数存储设备。

dtype: torch.dtype | None = None: 参数数据类型。

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

处理形状为 (batch_size, sequence_length, d_model) 的输入，并返回相同形状的张量。

注意：如上所述，在执行归一化前把输入上转换为 torch.float32，完成后再转换回原始数据类型。

测试时，实现适配器 [adapters.run_rmsnorm]，然后运行：

```
uv run pytest -k test_rmsnorm
```

3.4.2 逐位置前馈网络

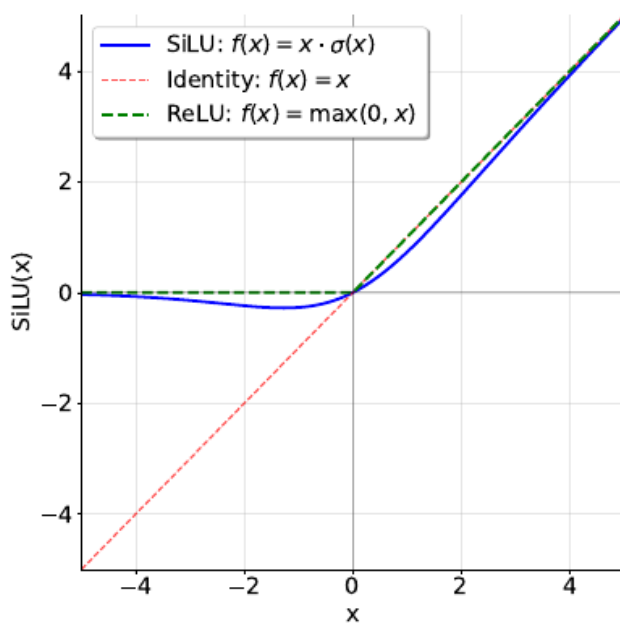


图 3: SiLU（又称 Swish）与 ReLU 激活函数的比较。

原始 Transformer 论文的前馈网络由两个线性变换组成，中间使用 ReLU 激活函数， $\text{ReLU}(x) = \max(0, x)$ ，见 A. Vaswani 等 [8] 第 3.3 节。在原始架构中，前馈网络内部层的维度通常是输入维度的 4 倍。

现代语言模型相较原始设计通常有两个主要变化：使用不同的激活函数，并引入门控机制。具体来说，本作业实现 Llama 3 [A. Grattafiori 等，2024] 和 Qwen 2.5 [A. Yang 等，2024] 等大语言模型采用的 **SwiGLU**。它把 SiLU，也常称为 Swish，与一种称为门控线性单元（GLU）的门控机制结合。按照 PaLM [A. Chowdhery 等，2022] 和 LLaMA [H. Touvron 等，2023] 以来多数现代大语言模型的做法，线性层也不使用偏置项。

SiLU 或 Swish 激活函数 [D. Hendrycks 等，2016；S. Elfwing 等，2017] 定义为：

$$\text{SiLU}(x) = x \cdot \sigma(x) = x / (1 + e^{\Lambda(-x)}) \quad (5)$$

如图 3 所示，SiLU 与 ReLU 相似，但它在零点处是平滑的。

门控线性单元（GLU）最初由 Y. N. Dauphin 等 [19] 定义。它是“一个经过 sigmoid 的线性变换”与“另一个线性变换”的逐元素乘积：

$$\text{GLU}(x, W1, W2) = \sigma(W1 \cdot x) \odot W2 \cdot x \quad (6)$$

其中 $\odot$ 表示逐元素乘法。据称，GLU 通过为梯度提供线性通路，同时保留非线性能力，可以“缓解深层架构中的梯度消失问题”。

把 SiLU/Swish 与 GLU 结合，就得到本作业前馈网络所使用的 SwiGLU：

$$\begin{aligned} \text{FFN}(x) &= \text{SwiGLU}(x, W1, W2, W3) \\ &= W2(\text{SiLU}(W1 \cdot x) \odot W3 \cdot x) \end{aligned} \quad (7)$$

其中：

$$x \in \mathbb{R}^{d_{\text{model}}}$$

$$W1, W3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$$

$$W2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$$

标准设置为 $d_{\text{ff}} = (8/3) d_{\text{model}}$

在具体实现中，为提高硬件效率，可以把 d_{ff} 舍入到附近的 64 的倍数。

N. Shazeer [20] 首次提出把 SiLU/Swish 与 GLU 结合，并通过实验证明，在语言建模任务上，SwiGLU 优于 ReLU 和不带门控的 SiLU 等基线。作业后面会比较 SwiGLU 与 SiLU。虽然我们给出了一些启发式解释，相关论文也提供了更多证据，但最好始终保持经验主义视角。Shazeer 论文中有一句如今广为流传的话：

“我们无法解释这些架构为什么似乎有效；和其他一切一样，我们把它们的成功归因于神的恩惠。”

问题（positionwise_feedforward）：实现逐位置前馈网络（2 分）

提交内容： 实现由 SiLU 激活函数与 GLU 组成的 SwiGLU 前馈网络。

注意：在这里，为保证数值稳定性，可以使用 `torch.sigmoid`。

实现中应把 d_{ff} 设为约 $(8/3) \times d_{\text{model}}$ ，并确保内部前馈层维度是 64 的倍数，以充分利用硬件。

测试时，实现适配器 `[adapters.run_swiglu]`，然后运行：

```
uv run pytest -k test_swiglu
```

3.4.3 相对位置嵌入

为了把位置信息注入模型，本作业实现旋转位置嵌入（Rotary Position Embeddings, RoPE）[J. Su 等, 2021]。

对 token 位置 i 上的某个 query 向量：

$$q^{\wedge}(i) = W_q \cdot x^{\wedge}(i) \in \mathbb{R}^d$$

应用成对旋转矩阵 R_i ：

$$q'^{\wedge}(i) = R_i \cdot q^{\wedge}(i) = R_i \cdot W_q \cdot x^{\wedge}(i)$$

R_i 会把嵌入元素对 $q^{\wedge}(i)_{(2k-1:2k)}$ 看成二维向量，并按角度旋转：

$$\theta_{(i,k)} = i / \Theta^{\wedge}((2k-2)/d), \quad k \in \{1, \dots, d/2\}$$

其中 Θ 是某个常数。由此，可把 R_i 看成大小为 $d \times d$ 的块对角矩阵，其第 k 个二维块为：

$$R_{k^i} = \begin{bmatrix} \cos(\theta_{(i,k)}) & \sin(\theta_{(i,k)}) \\ -\sin(\theta_{(i,k)}) & \cos(\theta_{(i,k)}) \end{bmatrix} \quad (8)$$

因此完整旋转矩阵是：

$$R_i = \text{diag}(R_{1^i}, R_{2^i}, \dots, R_{(d/2)^i}) \quad (9)$$

其中非对角位置都是 2×2 的零矩阵。

虽然可以显式构造完整的 $d \times d$ 矩阵，但好的实现应利用其结构，更高效地完成变换。由于我们只关心同一序列中 token 之间的相对旋转，计算出的 $\cos(\theta_{(i,k)})$ 和 $\sin(\theta_{(i,k)})$ 可以跨层、跨 batch 复用。

若希望优化，可让所有层引用同一个 RoPE 模块，并在初始化时使用 `self.register_buffer(persistent=False)` 创建预计算的二维 $\sin$ 、 $\cos$ 缓冲区，而不是使用 `nn.Parameter`，因为这些固定的正弦余弦值不应被学习。

对 key 向量 $k^{(j)}$ 也执行完全相同的旋转，只是使用对应的 R_j 。RoPE 层没有可学习参数。

问题 (rope)：实现 RoPE (2 分)

提交内容： 实现一个 `RotaryPositionalEmbedding` 类，对输入张量应用 RoPE。

建议接口：

```
def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None)
```

构造 RoPE 模块，并在需要时创建缓冲区。参数：

`theta: float`：RoPE 中的 θ 值。

`d_k: int`：query 和 key 向量维度。

`max_seq_len: int`：可能输入的最大序列长度。

`device: torch.device | None = None`：缓冲区存储设备。

```
def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor
```

处理形状为 $(\dots, \text{seq_len}, d_k)$ 的输入，返回相同形状的张量。x 可以具有任意数量的批次维度。假设 `token_positions` 的形状为 $(\dots, \text{seq_len})$ ，指定 x 在序列维度上的 token 位置。

应使用 token 位置沿序列维度切片可能已经预计算好的 $\cos$ 和 $\sin$ 张量。

测试时，实现 `[adapters.run_rope]`，并确保通过：

```
uv run pytest -k test_rope
```

3.4.4 缩放点积注意力

现在实现 A. Vaswani 等 [8] 第 3.2.1 节描述的缩放点积注意力。注意力定义会用到 softmax。Softmax 把未归一化的分数向量转换成归一化分布：

$$\text{softmax}(v)_i = \exp(v_i) / \sum_{(j=1)}^n \exp(v_j) \quad (10)$$

当 v_i 很大时， $\exp(v_i)$ 可能变成 `inf`，随后出现 `inf / inf = NaN`。注意到：给所有输入加上同一个常数 c ，softmax 的输出不变。可以利用这一性质提高数值稳定性。通常从 v 的所有元素中减去 v 的最大值，使新向量的最大元素变为 0。

问题 (softmax) : 实现 softmax (1 分)

提交内容： 编写一个函数，对张量应用 softmax。函数接收两个参数：一个张量和一个维度 i ，并沿输入张量的第 i 个维度应用 softmax。输出形状应与输入相同，但第 i 维上的值成为归一化概率分布。使用“减去第 i 维最大值”的技巧避免数值稳定性问题。

实现 [adapters.run_softmax]，并确保通过：

```
uv run pytest -k test_softmax_matches_pytorch
```

注意力运算可定义为：

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V \quad (11)$$

其中：

$$Q \in \mathbb{R}^{(n \times d_k)}$$

$$K \in \mathbb{R}^{(m \times d_k)}$$

$$V \in \mathbb{R}^{(m \times d_v)}$$

Q 、 K 、 V 都是该运算的输入，不是可学习参数。

掩码 (masking)： 有时需要屏蔽注意力运算的部分输出。掩码形状应为 $M \in \{\text{True}, \text{False}\}^{(n \times m)}$ ，布尔矩阵的第 i 行表示 query i 可以关注哪些 key。

按照惯例，也略显反直觉地，位置 (i, j) 为 True 表示 query i 会关注 key j ；为 False 表示不会关注。换句话说，值为 True 的 (i, j) 对允许“信息流动”。例如， 1×3 掩码 $[[\text{True}, \text{True}, \text{False}]]$ 表示唯一的 query 只关注前两个 key。

从计算角度看，使用掩码比在子序列上分别计算注意力高效得多。实现时，可以在 softmax 前的值 $QK^T / \sqrt{d_k}$ 中，对掩码为 False 的位置加上 $-\infty$ 。

问题 (scaled_dot_product_attention) : 实现缩放点积注意力 (5 分)

提交内容： 实现缩放点积注意力函数。实现应支持：

key 和 query 的形状为 $(\text{batch_size}, \dots, \text{seq_len}, d_k)$ 。

value 的形状为 $(\text{batch_size}, \dots, \text{seq_len}, d_v)$ 。

$\dots$ 表示任意数量的额外批次型维度。

返回形状为 $(\text{batch_size}, \dots, \text{seq_len}, d_v)$ 的输出。

关于批次型维度，见第 3.2 节。

实现还应支持用户可选提供的布尔掩码，形状为 $(\text{seq_len}, \text{seq_len})$ 。掩码为 True 的位置对应的注意力概率之和应为 1，掩码为 False 的位置注意力概率应为 0。

测试时，实现适配器 [adapters.run_scaled_dot_product_attention]：

```
uv run pytest -k test_scaled_dot_product_attention
```

该测试使用三阶输入张量；

```
uv run pytest -k test_4d_scaled_dot_product_attention
```

则使用四阶输入张量。

3.4.5 因果多头自注意力

本作业按照 A. Vaswani 等 [8] 第 3.2.2 节实现多头自注意力。数学上，多头注意力定义为：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \quad (12)$$

其中：

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (13)$$

Q_i 、 K_i 、 V_i 分别是 Q 、 K 、 V 在嵌入维度上第 $i \in \{1, \dots, h\}$ 个大小为 d_k 或 d_v 的切片； Attention 是第 3.4.4 节定义的缩放点积注意力。

由此可得到多头自注意力：

$$\begin{aligned} \text{MultiHeadSelfAttention}(x) \\ = W_0 \text{MultiHead}(W_Q x, W_K x, W_V x) \end{aligned} \quad (14)$$

可学习参数为：

$$W_Q \in \mathbb{R}^{(h \, d_k \times d_{\text{model}})}$$

$$W_K \in \mathbb{R}^{(h \, d_k \times d_{\text{model}})}$$

$$W_V \in \mathbb{R}^{(h \, d_v \times d_{\text{model}})}$$

$$W_0 \in \mathbb{R}^{(d_{\text{model}} \times h \, d_v)}$$

由于多头注意力会对 Q 、 K 、 V 进行切片，可以把 W_Q 、 W_K 、 W_V 看成沿输出维度为各注意力头分隔开的矩阵。实现完成后，key、value 和 query 三个投影总共应只使用三次矩阵乘法。⁵

因果掩码

实现必须阻止模型关注序列中的未来 token。若模型输入 token 序列 $t_1, \dots, t_n$ ，并希望计算前缀 $t_1, \dots, t_i$ 的下一个 token 预测，其中 $i < n$ ，模型就不能访问位置 $t_{(i+1)}, \dots, t_n$ 的 token 表示。推理生成时，这些未来 token 本来就不存在；若训练时能访问，它们会泄漏真实下一个 token 的身份，使语言模型预训练目标变得毫无难度。

对于长度为 n 的输入序列，朴素做法是对 n 个不同前缀分别运行一次多头自注意力，共运行 n 次。更高效的做法是使用因果注意力掩码，使位置 i 只能关注序列中满足 $j \leq i$ 的位置。

可以使用 `torch.triu` 或广播后的索引比较构造掩码，并利用第 3.4.4 节的缩放点积注意力已经支持掩码这一事实。

应用 RoPE

RoPE 应作用于 query 和 key 向量，而不作用于 value 向量。注意力头维度应当作为批次维度处理，因为多头注意力对每个头独立计算注意力。这意味着对每个头的 query、key 向量应用完全相同的 RoPE 旋转。

⁵ 进阶目标：尝试把 key、query 和 value 投影合并到一个权重矩阵中，使它们只需一次矩阵乘法。

问题 (multihead_self_attention) : 实现因果多头自注意力 (5 分)

提交内容： 把因果多头自注意力实现为 `torch.nn.Module`。至少支持以下构造参数：

`d_model`: int: Transformer 块输入的维度。

`num_heads`: int: 多头自注意力中的注意力头数量。

按照 A. Vaswani 等 [8], 设置：

```
d_k = d_v = d_model / h
```

测试时，实现适配器 `[adapters.run_multihead_self_attention]`，然后运行：

```
uv run pytest -k test_multihead_self_attention
```

3.5 完整 Transformer 语言模型

先把 Transformer 块组装起来，可参考图 2。一个 Transformer 块包含两个子层：一个用于多头自注意力，另一个用于 SwiGLU 前馈网络。在每个子层中，先执行 RMSNorm，再执行主要运算 MHA 或 FFN，最后加上残差连接。

具体来说，Transformer 块的前半部分，即第一个子层，应当从输入 x 生成输出 y ：

```
y = x + MultiHeadSelfAttention(RMSNorm(x))
```

 (15)

问题 (transformer_block) : 实现 Transformer 块 (3 分)

按照第 3.4 节的描述和图 2，实现 pre-norm Transformer 块。至少支持以下参数：

`d_model`: int: Transformer 块输入维度。

`num_heads`: int: 多头自注意力中的注意力头数量。

`d_ff`: int: 逐位置前馈网络内部层维度。

测试时，实现适配器 `[adapters.run_transformer_block]`，然后运行：

```
uv run pytest -k test_transformer_block
```

提交内容： 能通过提供测试的 Transformer 块代码。

接下来按照图 1 的高层结构把各模块组合起来。遵循第 3.1 节对嵌入层的说明，把嵌入输出输入到 `num_layers` 个 Transformer 块中，再经过最终层归一化与 LM head，得到词表上的未归一化分布，也就是 logits。

问题 (transformer_lm) : 实现 Transformer 语言模型 (3 分)

现在把所有内容组合起来。按照第 3.1 节和图 1 实现 Transformer 语言模型。除前面 Transformer 块的构造参数外，至少还要支持：

`vocab_size`: int: 词表大小，用于确定 token 嵌入矩阵维度。

`context_length`: int: 最大上下文长度，用于确定 RoPE 的 `sin`、`cos` 缓冲区维度。

`num_layers`: int: Transformer 块数量。

测试时，实现适配器 `[adapters.run_transformer_lm]`，然后运行：

```
uv run pytest -k test_transformer_lm
```

提交内容：能通过上述测试的 Transformer LM 模块。

资源核算

理解 Transformer 的各部分如何消耗计算量和内存非常重要。下面进行基础的 FLOPs 核算。

Transformer 中绝大多数 FLOPs 来自矩阵乘法，因此核心方法很简单：

列出 Transformer 前向传播中的全部矩阵乘法。

把每次矩阵乘法转换成所需 FLOPs。

第二步会用到以下规则：

规则：给定 $A \in \mathbb{R}^{(m \times n)}$ 与 $B \in \mathbb{R}^{(n \times p)}$ ，矩阵乘积 AB 需要 $2mnp$ FLOPs。

原因是 $(AB)[i, j] = A[i, :] \cdot B[:, j]$ ，该点积需要 n 次加法和 n 次乘法，共 $2n$ FLOPs。矩阵 AB 有 $m \times p$ 个元素，因此总 FLOPs 为 $(2n)(mp) = 2mnp$ 。

在做下一题前，最好遍历 Transformer 块和完整 Transformer LM 的每个组件，列出全部矩阵乘法及其 FLOPs 开销。

问题 (transformer_accounting)：Transformer LM 资源核算 (5 分)

(a) 考虑一个采用本作业架构、规模与 GPT-2 XL 相当的模型，配置如下：

```
vocab_size:    50,257
context_length: 1,024
num_layers:    48
d_model:      1,600
num_heads:    25
d_ff:         4,288
```

这里 $d_{ff} = 4,288$ ，是 $(8/3) \times 1,600$ 附近最近的 64 的倍数。

使用该配置构建模型时，有多少可训练参数？若每个参数使用单精度浮点表示，仅加载模型需要多少内存？

提交内容：一到两句话。

(b) 找出完成这个 GPT-2 XL 规模模型一次前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs？假设输入序列包含 `context_length` 个 token。

提交内容：矩阵乘法列表，每项附描述，并给出总 FLOPs。

(c) 根据上述分析，模型的哪些部分需要最多 FLOPs？

提交内容：一到两句话。

(d) 对 GPT-2 small、medium 和 large 重复上述分析：

GPT-2 small: 12 层, $d_{model} = 768$, 12 个头。

GPT-2 medium: 24 层, $d_{model} = 1024$, 16 个头。

GPT-2 large: 36 层, $d_{model} = 1280$, 20 个头。

随着模型规模增大，Transformer LM 的哪些部分在总 FLOPs 中所占比例变大或变小？

提交内容： 对每个模型给出各组件 FLOPs 占总前向 FLOPs 的比例，并用一到两句话说明模型规模变化如何改变各组件的相对 FLOPs。

(e) 对 GPT-2 XL，把上下文长度增加到 16,384。一次前向传播的总 FLOPs 如何变化？各模型组件的相对 FLOPs 贡献如何变化？

提交内容： 一到两句话。

4 训练 Transformer 语言模型

目前，我们已经有了预处理数据所需的分词器，以及 Transformer 模型本身。接下来还需要构建支持训练的全部代码，包括：

损失函数： 定义交叉熵损失。

优化器： 定义用于最小化损失的 AdamW 优化器。

训练循环： 构建加载数据、保存检查点和管理训练过程的基础设施。

4.1 交叉熵损失

回顾一下，对于每个长度为 $m + 1$ 的序列 x ，以及 $i = 1, \dots, m$ ，Transformer 语言模型定义了条件分布：

$$p_{\theta}(x_{(i+1)} \mid x_{(1:i)})$$

给定由若干长度为 $m + 1$ 的序列组成的训练集 D ，标准交叉熵，也就是负对数似然损失，定义为：

$$\ell(\theta; D) = (1 / (|D| \cdot m)) \cdot \sum_{(x \in D)} \sum_{i=1}^m -\log p_{\theta}(x_{(i+1)} \mid x_{(1:i)}) \quad (16)$$

注意，一次 Transformer 前向传播就能同时得到所有 $i = 1, \dots, m$ 对应的 $p_{\theta}(x_{(i+1)} \mid x_{(1:i)})$ 。

具体而言，Transformer 在每个位置 i 计算一个 logits 向量 $o_i \in \mathbb{R}^{(\text{vocab_size})}$ ，于是：⁶

$$\begin{aligned} p(x_{(i+1)} \mid x_{(1:i)}) &= \text{softmax}(o_i)[x_{(i+1)}] \\ &= \exp(o_i[x_{(i+1)}]) / \sum_{a=1}^{\text{vocab_size}} \exp(o_i[a]) \end{aligned} \quad (17)$$

交叉熵损失通常就是针对 logits 向量 $o_i \in \mathbb{R}^{(\text{vocab_size})}$ 与目标 token $x_{(i+1)}$ 定义的。⁷

与 softmax 一样，实现交叉熵时也必须谨慎处理数值问题。

问题（cross_entropy）：实现交叉熵（1 分）

提交内容： 编写函数计算交叉熵损失。输入为预测 logits o_i 和目标 token $x_{(i+1)}$ ，输出：

$$\ell_i = -\log \text{softmax}(o_i)[x_{(i+1)}]$$

函数应满足：

为保证数值稳定性，减去最大元素。

⁶ $o_i[k]$ 表示向量 o_i 在索引 k 处的值。

⁷ 这对应于真实 token $x_{(i+1)}$ 上的 Dirac delta 分布与预测分布 $\text{softmax}(o_i)$ 之间的交叉熵。

尽可能约掉 $\log$ 与 $\exp$ ，避免不必要计算。

支持任意额外 batch 维度，并返回整个 batch 上的平均值。与第 3.2 节相同，假设所有批次型维度都位于词表维度之前。

实现 `[adapters.run_cross_entropy]`，然后运行：

```
uv run pytest -k test_cross_entropy
```

困惑度

交叉熵足以用于训练，但评估模型时还希望报告困惑度。对于长度为 m 、各位置交叉熵损失为 $\ell_1, \dots, \ell_m$ 的序列：

$$\text{perplexity} = \exp\left(\frac{1}{m} \cdot \sum_{i=1}^m \ell_i\right) \quad (18)$$

4.2 SGD 优化器

有了损失函数后，开始研究优化器。最简单的基于梯度的优化器是随机梯度下降（SGD）。先随机初始化参数 θ_0 ，然后对每一步 $t = 0, \dots, T - 1$ 执行：

$$\theta_{(t+1)} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t) \quad (19)$$

其中， B_t 是从数据集 D 中随机采样的一个 batch；学习率 α_t 和 batch 大小 $|B_t|$ 都是超参数。

4.2.1 在 PyTorch 中实现 SGD

本作业通过继承 PyTorch 的 `torch.optim.Optimizer` 类实现优化器。一个 `Optimizer` 子类必须实现两个方法：

```
def __init__(self, params, ...)
```

用于初始化优化器。`params` 是待优化参数的集合，也可能是一组参数组，使用户能对模型不同部分使用不同的超参数，例如不同学习率。务必把 `params` 传给基类构造函数，基类会保存这些参数供 `step` 使用。

还可以根据具体优化器接收额外参数，例如学习率，并以字典形式传给基类构造函数；字典键是你为这些参数选定的字符串名称。

```
def step(self)
```

用于执行一次参数更新。训练循环会在反向传播后调用它，因此此时可以访问上一批数据产生的梯度。该方法应遍历每个参数张量 p ，并原地修改它，也就是更新 `p.data`。若 `p.grad` 存在，它就是损失对该参数的梯度张量。

PyTorch 优化器 API 有一些细节，下面通过示例说明。为了让示例更丰富，我们实现一个略作修改的 SGD：学习率会随训练衰减，从初始学习率 α 开始，步长逐渐减小：

$$\theta_{(t+1)} = \theta_t - \alpha / \sqrt{t + 1} \cdot \nabla L(\theta_t; B_t) \quad (20)$$

以下是该 SGD 变体作为 PyTorch `Optimizer` 的实现：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
```

```

if lr < 0:
    raise ValueError(f"Invalid learning rate: {lr}")
defaults = {"lr": lr}
super().__init__(params, defaults)

def step(self, closure: Optional[Callable] = None):
    loss = None if closure is None else closure()
    for group in self.param_groups:
        lr = group["lr"] # 读取学习率
        for p in group["params"]:
            if p.grad is None:
                continue
            state = self.state[p] # 获取与 p 关联的状态
            t = state.get("t", 0) # 读取迭代次数; 不存在时设为 0
            grad = p.grad.data # 读取损失对 p 的梯度
            p.data -= lr / math.sqrt(t + 1) * grad # 原地更新权重张量
            state["t"] = t + 1 # 迭代次数加 1
    return loss

```

在 `__init__` 中，把参数和默认超参数传给基类构造函数。参数可能被组织成多组，每组可有不同超参数。如果传入的只是单个 `torch.nn.Parameter` 集合，基类构造函数会创建一个参数组，并为其分配默认超参数。

在 `step` 中，先遍历参数组，再遍历组中的参数，并应用公式 20。这里把迭代次数保存为每个参数的状态：先读取该值，用于梯度更新，再把它加 1。

API 允许用户传入一个可调用的 `closure`，以便在优化器步骤之前重新计算损失。本作业不会用到这一功能，但为了符合 API，示例仍加入了该参数。

可以使用下面的最小训练循环测试该优化器：

```

weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # 清空所有可学习参数的梯度
    loss = (weights**2).mean() # 计算标量损失
    print(loss.cpu().item())
    loss.backward() # 反向传播并计算梯度
    opt.step() # 执行优化器步骤

```

这就是训练循环的典型结构：每次迭代计算损失，并执行一次优化器更新。训练语言模型时，可学习参数来自模型；在 PyTorch 中，`m.parameters()` 会返回这些参数。损失在随机采样的一批数据上计算，但训练循环的基本结构相同。

问题 (learning_rate_tuning)：调节学习率 (1 分)

学习率是影响训练最显著的超参数之一。请在上述玩具 SGD 示例中，分别使用另外三个学习率 `1e1`、`1e2` 和 `1e3`，每种只训练 10 次迭代。观察各学习率下的损失会发生什么：下降更快、下降更慢，还是发散，也就是训练过程中反而增大？

提交内容：一到两句话，说明观察到的行为。

4.3 AdamW

现代语言模型通常不使用 SGD，而是使用更复杂的优化器。近年来大部分优化器都是 Adam [D. P. Kingma 等, 2015] 的变体。本作业使用近期工作中广泛采用的 AdamW [I. Loshchilov 等, 2019]。

AdamW 对 Adam 进行了修改：通过加入权重衰减改善正则化，也就是每次迭代都把参数轻微拉向 0；同时，权重衰减与梯度更新解耦。本作业按照 I. Loshchilov 等 [23] 的算法 2 实现 AdamW。

AdamW 是有状态的：对每个参数，它都会维护一阶矩和二阶矩的运行估计。因此，AdamW 以额外内存换取更好的稳定性和收敛性。

除学习率 α 外，AdamW 还有一对控制矩估计更新的超参数 (β_1, β_2) ，以及权重衰减率 λ 。典型应用常将 (β_1, β_2) 设为 $(0.9, 0.999)$ ；但 LLaMA [H. Touvron 等, 2023] 和 GPT-3 [T. B. Brown 等, 2020] 等大语言模型经常使用 $(0.9, 0.95)$ 。

算法如下，其中 ϵ 是一个很小的数，例如 10^{-8} ，用于在 v 极小时提高数值稳定性。

算法 1: AdamW 优化器

```
1  init( $\theta$ )                初始化可学习参数
2   $m \leftarrow 0$           初始化一阶矩向量，形状与  $\theta$  相同
3   $v \leftarrow 0$           初始化二阶矩向量，形状与  $\theta$  相同
4  for  $t = 1, \dots, T$  do
5      采样一批数据  $B_t$ 
6       $g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$     计算损失梯度
7       $\alpha_t \leftarrow \alpha \cdot \text{sqrt}(1 - \beta_2^t) / (1 - \beta_1^t)$ 
                                                计算第  $t$  次迭代的修正学习率
8       $\theta \leftarrow \theta - \alpha \lambda \theta$     应用权重衰减
9       $m \leftarrow \beta_1 m + (1 - \beta_1) g$         更新一阶矩估计
10      $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$       更新二阶矩估计
11      $\theta \leftarrow \theta - \alpha_t \cdot m / (\text{sqrt}(v) + \epsilon)$  应用经矩估计调整的权重更新
12 end for
```

注意， t 从 1 开始。现在实现该优化器。

问题 (adamw)：实现 AdamW (2 分)

提交内容：把 AdamW 实现为 `torch.optim.Optimizer` 的子类。构造函数 `__init__` 应接收学习率 α ，以及 β 、 ϵ 、 λ 等超参数。

基类 `Optimizer` 提供了字典 `self.state`，它把 `nn.Parameter` 对象映射到保存该参数相关信息的字典。对 AdamW 来说，其中应保存矩估计等状态。

实现 `[adapters.get_adamw_cls]`，并确保通过：

```
uv run pytest -k test_adamw
```

问题 (adamw_accounting)：AdamW 训练资源核算 (2 分)

计算运行 AdamW 所需的内存和计算量。假设每个张量都使用 `float32`。

(a) 运行 AdamW 的峰值内存是多少？请按以下部分拆分：

参数

激活值

梯度

优化器状态

用 batch_size 和模型超参数 vocab_size 、 context_length 、 num_layers 、 d_model 、 num_heads 表示答案。假设：

$$\text{d_ff} = (8/3) \times \text{d_model}$$

为简化激活值内存计算，只考虑以下组件：

Transformer 块

RMSNorm

多头自注意力子层：QKV 投影、 QK^T 矩阵乘法、softmax、value 加权和、输出投影

逐位置前馈网络（SwiGLU）： W_1 、 W_2 、门控分支上的 SiLU、逐元素乘法、 W_3

最终 RMSNorm

输出嵌入

logits 上的交叉熵

提交内容： 分别给出参数、激活值、梯度、优化器状态及总内存的代数表达式。

(b) 将答案代入 GPT-2 XL 规模模型，使表达式只依赖 batch_size 。在 80 GB 内存中能容纳的最大 batch size 是多少？

提交内容： 形如 $a \cdot \text{batch_size} + b$ 的表达式，其中 a 、 b 为数值；并给出最大 batch size。

(c) 运行 AdamW 的一个训练步骤需要多少 FLOPs？

提交内容： 代数表达式及简短说明。

(d) 模型 FLOPs 利用率（Model FLOPs Utilization, MFU）定义为：实际观察到的吞吐量，也就是每秒 token 数，相对于硬件理论峰值 FLOPs 吞吐量的比例 [A. Chowdhery 等，2022]。

NVIDIA H100 GPU 的“float32”运算理论峰值为每秒 495 teraFLOP；这里实际指 TensorFloat-32，原文称它实际上是“bfloat19”。假设能达到 50% MFU，在单张 H100 上，以 batch size 1024 训练 GPT-2 XL 共 400K 步，需要多长时间？按照 J. Kaplan 等 [25] 与 J. Hoffmann 等 [26]，假设反向传播 FLOPs 是前向传播的两倍。

提交内容： 所需小时数及简短推导。

4.4 学习率调度

能让损失下降最快的学习率通常会随训练阶段改变。训练 Transformer 时，常使用学习率调度：训练初期采用较大学习率，以快速更新；随着训练进行，再逐渐衰减到较小值。⁸

本作业实现训练 LLaMA [H. Touvron 等，2023] 时使用的余弦退火调度。

⁸ 有时也会采用让学习率重新上升，也就是“重启”的调度，以帮助模型越过局部极小值。

调度器本质上只是一个函数：输入当前步数 t 和其他相关参数，例如初始、最终学习率，返回第 t 步梯度更新时应使用的学习率。最简单的调度是常数函数，无论 t 是多少，都返回相同学习率。

余弦退火学习率调度接收：

当前迭代 t

最大学习率 $\alpha_{\max}$

最小或最终学习率 $\alpha_{\min}$

预热迭代数 T_w

余弦退火结束迭代 T_c

第 t 次迭代的学习率定义为：

预热阶段，若 $t < T_w$ ：

$$\alpha_t = (t / T_w) \alpha_{\max}$$

余弦退火阶段，若 $T_w \leq t \leq T_c$ ：

$$\alpha_t = \alpha_{\min} + 1/2 \cdot [1 + \cos((t - T_w)/(T_c - T_w) \cdot \pi)] \cdot (\alpha_{\max} - \alpha_{\min})$$

退火结束后，若 $t > T_c$ ：

$$\alpha_t = \alpha_{\min}$$

问题 (learning_rate_schedule)：实现带预热的余弦学习率调度 (1 分)

编写函数，输入 t 、 $\alpha_{\max}$ 、 $\alpha_{\min}$ 、 T_w 、 T_c ，按上述调度返回学习率 α_t 。然后实现

`[adapters.get_lr_cosine_schedule]`，并确保通过：

```
uv run pytest -k test_get_lr_cosine_schedule
```

4.5 梯度裁剪

训练过程中，偶尔会遇到产生巨大梯度的训练样本，从而导致训练不稳定。实践中常用梯度裁剪缓解这一问题：每次反向传播后、优化器更新前，限制梯度范数。

给定所有参数的梯度 g ，计算其 L2 范数 $\|g\|_2$ 。若范数小于最大值 M ，保持 g 不变；否则，把 g 缩小为原来的：

$$M / (\|g\|_2 + \epsilon)$$

其中加入一个很小的 ϵ ，例如 10^{-6} ，用于数值稳定性。裁剪后范数会略小于 M 。

问题 (gradient_clipping)：实现梯度裁剪 (1 分)

编写函数实现梯度裁剪。函数接收参数列表与最大 L2 范数，并原地修改每个参数的梯度。使用 PyTorch 默认值 $\epsilon = 10^{-6}$ 。

实现适配器 `[adapters.run_gradient_clipping]`，并确保通过：

```
uv run pytest -k test_gradient_clipping
```

5 训练循环

现在把此前构建的主要组件组合起来：分词后的数据、模型和优化器。

5.1 数据加载器

分词后的数据，例如你在 `tokenizer_experiments` 中准备的数据，是一个单独的 token 序列：

```
x = (x_1, ..., x_n)
```

即使源数据由独立文档组成，例如不同网页或源代码文件，常见做法仍是把它们拼接成一个 token 序列，并在文档之间加入分隔符，例如 `<|endoftext|>`。

数据加载器把这个长序列转换成一系列 batch。每个 batch 包含 B 个长度为 m 的输入序列，以及对应的、同样长度为 m 的下一个 token 目标序列。例如，当 $B = 1$ 、 $m = 3$ 时：

```
([x_2, x_3, x_4], [x_3, x_4, x_5])
```

就是一个可能的 batch。

这种数据加载方式能从多方面简化训练：

任意 $1 \leq i \leq n - m$ 都能产生一个合法训练序列，因此采样非常简单。

所有训练序列长度相同，无需 padding；这能提高硬件利用率，也方便增大 batch size B 。

采样训练数据时无需把完整数据集装入内存，因此可以处理本来无法完整放入内存的大型数据集。

问题（data_loading）：实现数据加载（2 分）

提交内容： 编写函数，输入：

一个 NumPy 数组 x ，其中是整数 token ID

`batch_size`

`context_length`

PyTorch 设备字符串，例如 `'cpu'` 或 `'cuda:0'`

返回一对张量：随机采样的输入序列，以及对应的下一个 token 目标。两个张量形状都应为 $(batch_size, context_length)$ ，内容为 token ID，并放到指定设备上。

实现测试适配器 `[adapters.run_get_batch]`，然后运行：

```
uv run pytest -k test_get_batch
```

低资源提示：在 CPU 或 Apple Silicon 上加载数据

如果准备在 CPU 或 Apple Silicon 上训练语言模型，需要把数据移动到正确设备；后续模型也应使用相同设备。CPU 使用设备字符串 `'cpu'`，Apple Silicon 的 M 系列芯片使用 `'mps'`。

MPS 相关资料：

- <https://docs.pytorch.org/docs/stable/mps.html>

- <https://docs.pytorch.org/docs/stable/notes/mps.html>

- <https://developer.apple.com/documentation/metalperformanceshaders>

如果数据集过大，无法放入内存，可以使用 Unix 系统调用 `mmap`，它把磁盘文件映射到虚拟内存，并在访问相应内存位置时惰性加载文件内容。这样就能“假装”整个数据集都在内存中。

NumPy 通过 `np.memmap` 实现这一机制；如果数组最初通过 `np.save` 保存，也可以在 `np.load` 中使用 `mmap_mode='r'`。它会返回一个类似 NumPy 数组的对象，只在访问元素时按需加载。

训练期间从数据集，也就是 NumPy 数组中采样时，务必以 `memory-mapped` 模式加载：根据保存格式，使用 `np.memmap` 或 `np.load(..., mmap_mode='r')`。还要指定与所加载数组一致的 `dtype`。建议显式验证映射后的数据是否正确，例如确认其中没有超过预期词表大小的值。

5.2 检查点

除了加载数据，训练过程中还需要保存模型。运行训练任务时，经常需要从中途停止的位置恢复，例如任务超时、机器故障等。即使训练正常完成，也可能希望之后访问中间模型，以便事后研究训练动态，或从不同训练阶段的模型中采样。

一个检查点必须包含恢复训练所需的全部状态。至少要保存模型权重。若使用 AdamW 等有状态优化器，还要保存优化器状态，例如 AdamW 的矩估计。最后，为了恢复学习率调度，还要知道训练停止时的迭代次数。

PyTorch 对这些操作提供了便利支持：

每个 `nn.Module` 都有 `state_dict()`，返回包含全部可学习权重的字典。

可用 `load_state_dict()` 恢复这些权重。

每个 `torch.optim.Optimizer` 也提供相同机制。

`torch.save(obj, dest)` 可以把对象写入文件路径或类文件对象。`obj` 可以是包含张量、整数等普通 Python 对象的字典。

`torch.load(src)` 可以把它重新加载到内存。

问题（checkpointing）：实现模型检查点（1分）

实现以下两个函数。

```
def save_checkpoint(model, optimizer, iteration, out)
```

把模型、优化器和迭代次数的全部状态写入类文件对象 `out`。可以通过模型和优化器的 `state_dict` 获取相关状态，再用 `torch.save(obj, out)` 写出。PyTorch 在此既支持路径，也支持类文件对象。

典型做法是把 `obj` 设为字典，但也可以使用其他格式，只要之后能正确加载。

参数：

`model`: `torch.nn.Module`

`optimizer`: `torch.optim.Optimizer`

`iteration`: `int`

`out`: `str` | `os.PathLike` | `typing.BinaryIO` | `typing.IO[bytes]`

```
def load_checkpoint(src, model, optimizer)
```

从 `src`，即路径或类文件对象中加载检查点，并恢复模型与优化器状态。函数应返回检查点中保存的迭代次数。

可以用 `torch.load(src)` 读取 `save_checkpoint` 保存的内容，再调用模型和优化器的 `load_state_dict` 恢复原状态。

参数：

`src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

`model: torch.nn.Module`

`optimizer: torch.optim.Optimizer`

实现 `[adapters.run_save_checkpoint]` 和 `[adapters.run_load_checkpoint]`，并确保通过：

```
uv run pytest -k test_checkpointing
```

5.3 训练循环

现在终于可以把所有组件组合进主训练脚本。最好让启动不同超参数的训练任务变得很容易，例如把超参数设计成命令行参数。后续为了研究不同选择对训练的影响，你会多次启动不同训练任务。

问题（`training_together`）：整合全部组件（4 分）

提交内容： 编写脚本，运行训练循环，在用户提供的输入上训练模型。建议训练脚本至少支持：

配置并控制各种模型和优化器超参数。

使用 `np.memmap` 以内存高效方式加载大型训练集和验证集。

把检查点序列化到用户提供的路径。

定期记录训练与验证性能，例如输出到控制台或 `Weights & Biases` 等外部服务。⁹

6 生成文本

现在已经可以训练模型，最后还需要让模型生成文本。

语言模型接收一批可能带 `batch` 的、长度为 `sequence_length` 的整数序列，输出大小为 `(sequence_length, vocab_size)` 的矩阵。序列中每个位置对应一个预测下一个 `token` 的概率分布。下面编写若干函数，把这些分布转化为新序列的采样方案。

Softmax

按照标准惯例，语言模型直接输出最终线性层的结果，也就是 `logits`。因此，需要使用公式 10 中的 `softmax` 把它转换成归一化概率。

⁹ `wandb.ai`。

解码

为了让模型生成文本，也就是解码，我们向模型提供一个前缀 token 序列，即“提示词”或 prompt，让模型输出词表上的概率分布，预测序列中的下一个 token。随后从该词表分布中采样，确定下一个输出 token。

具体而言，一步解码接收序列 $x_{(1:t)}$ ，并按以下方式返回 token $x_{(t+1)}$ ：

$$P(x_{(t+1)} = i \mid x_{(1:t)}) = \exp(v_i) / \sum_j \exp(v_j) \quad (21)$$

其中：

$$v = \text{TransformerLM}(x_{(1:t)})_t \in \mathbb{R}^{\text{vocab_size}} \quad (22)$$

TransformerLM 接收长度为 `sequence_length` 的序列，输出大小为 $(\text{sequence_length}, \text{vocab_size})$ 的矩阵。由于要预测位置 t 后面的下一个 token，我们取矩阵的最后一个元素，也就是最后一个位置对应的词表 logits。

基础解码器会反复从这些单步条件分布中采样，并把前一步生成的 token 追加到下一步的输入，直到生成序列结束 token `<|endoftext|>`，或达到用户指定的最大生成 token 数。

解码技巧

本作业训练的是小模型，小模型有时会生成质量很低的文本。下面两个简单技巧可以改善结果。

温度缩放

温度缩放使用温度参数 τ 修改 softmax：

$$\begin{aligned} \text{softmax}(v, \tau)_i \\ = \exp(v_i / \tau) / \sum_{(j=1)}^{\text{vocab_size}} \exp(v_j / \tau) \end{aligned} \quad (23)$$

当 $\tau \rightarrow 0$ 时， v 中最大元素会占据主导，softmax 输出趋近于集中在最大元素上的 one-hot 向量。

Nucleus / top-p 采样

另一种技巧是 nucleus sampling，也称 top-p 采样，它通过截断低概率 token 修改采样分布。

设 q 是经过可能带温度缩放的 softmax 后得到的、大小为 `vocab_size` 的概率分布。超参数为 p 的 nucleus sampling 按以下方式生成下一个 token：

$$P(x_{(t+1)} = i \mid q) = \begin{cases} q_i / \sum_{(j \in V(p))} q_j, & \text{若 } i \in V(p) \\ 0, & \text{否则} \end{cases} \quad (24)$$

其中 $V(p)$ 是满足以下条件的最小索引集合：

$$\sum_{(j \in V(p))} q_j \geq p$$

可以先按概率从大到小排序，再从最大概率的词表项开始选取，直到累计概率达到目标 p 。

问题（decoding）：解码（3分）

提交内容： 实现一个从语言模型解码的函数。建议支持：

对用户提供的 prompt 生成补全：输入某个 $x_{(1:t)}$ ，持续采样，直到生成 `<|endoftext|>`。

允许用户控制最大生成 token 数。

根据用户指定的温度，对预测的下一个 token 分布应用 softmax 温度缩放，再进行采样。

根据用户指定的阈值执行 top-p 采样，也称 nucleus sampling [A. Holtzman 等，2020]。

7 实验

现在把全部内容组合起来，在预训练数据集上训练小型语言模型。

7.1 如何运行实验及提交内容

理解 Transformer 各架构组件设计依据的最佳方式，就是亲自修改模型并运行实验。动手经验无可替代。

因此，你需要能够**快速、稳定地开展实验，并记录所做工作**：

为了快速实验，我们会在一个小规模模型上运行大量实验。模型总参数约 17M，数据集使用简单的 TinyStories。

为了保证实验的一致性，你需要系统地消融组件、改变超参数。

为了保留记录，你需要提交实验日志，以及每项实验对应的学习曲线。

为了能够提交损失曲线，务必定期评估验证损失，并同时记录训练步数和墙钟时间。Weights & Biases 等日志基础设施可能会有帮助。

问题（experiment_log）：实验日志（3 分）

为训练与评估代码创建实验跟踪基础设施，使你能够按梯度更新步数与墙钟时间跟踪实验和损失曲线。

提交内容：实验日志基础设施代码，以及本节后续作业题对应的实验日志，也就是记录你尝试过的所有内容的文档。

7.2 TinyStories

我们先使用一个非常简单的数据集 TinyStories [R. Eldan 等，1]。模型在该数据集上训练很快，同时能观察到一些有趣行为。数据集获取说明见第 1 节。下面给出一个样本。

示例（tinystories_example）：TinyStories 样本

从前，有一个名叫本的小男孩。本喜欢探索周围的世界。他见过许多令人惊叹的东西，比如商店里陈列的漂亮花瓶。

有一天，本走过商店时，遇到了一个非常特别的花瓶。他一看到它就惊呆了！他说：“哇，这真是一个很棒的花瓶！我可以买下它吗？”店主笑着说：“当然可以。你可以把它带回家，让所有朋友看看它有多棒！”

于是本把花瓶带回家，他为此非常自豪。他把朋友们叫来，向他们展示这个神奇的花瓶。所有朋友都觉得花瓶很漂亮，不敢相信本竟然这么幸运。这就是本在商店里发现一个神奇花瓶的故事！

7.2.1 超参数调节

课程组会提供一些最基本的起始超参数，并要求你自行寻找其余效果较好的设置。

词表大小：10,000。 典型词表大小通常在几万到几十万之间。你应尝试改变它，观察词表和模型行为如何变化。

上下文长度：256。 TinyStories 这类简单数据集可能不需要很长的序列，但之后使用 OpenWebText 时，可能需要调整。尝试改变上下文长度，观察它对单次迭代运行时间和最终困惑度的影响。

d_model：512。 这略小于许多小型 Transformer 论文中使用的 768 维，因此训练更快。

d_ff：1344。 约等于 $(8/3) \times d_{\text{model}}$ ，同时是 64 的倍数，有利于 GPU 性能。

RoPE 的 theta 参数 θ ：10000。

层数与注意力头数：4 层、16 个头。 该配置大约有 17M 个不含嵌入的参数，属于较小的 Transformer。

处理的 token 总数：327,680,000。 你的：

```
batch size × 总训练步数 × 上下文长度
```

应大致等于该值。

你需要通过反复试验，为以下超参数找到较好的默认值：学习率、学习率预热、其他 AdamW 超参数 β_1 、 β_2 、 ϵ ，以及权重衰减。D. P. Kingma 等 [22] 中可以找到这些超参数的一些典型选择。

7.2.2 整合运行

现在可以组合所有内容：获得训练好的 BPE 分词器，对训练数据集进行分词，再使用自己编写的训练循环进行训练。

重要说明： 如果实现正确且高效，上述超参数在一张 B200 GPU 上应约需 20 到 30 分钟。如果运行时间明显更长，请检查：

数据加载是否成为瓶颈。

检查点保存是否成为瓶颈。

验证损失计算是否成为瓶颈。

实现是否正确进行了批处理。

7.2.3 调试模型架构的技巧

强烈建议熟悉 IDE 内置调试器，例如 VSCode 或 Zed。与用 `print` 语句调试相比，它会节省很多时间。若使用纯文本编辑器，可以使用 `ipdb` 等工具。

调试模型架构时，还可采用以下良好实践：

开发任何神经网络架构时，一个常见的第一步是让模型在单个 minibatch 上过拟合。如果实现正确，训练损失应能迅速降到接近 0。

在模型不同组件中设置断点，检查中间张量形状是否符合预期。

监控激活值、模型权重和梯度的范数，确保它们没有爆炸或消失。

问题 (learning_rate)：调节学习率 (2 B200 小时, 3 分)

学习率是最重要的超参数之一。以训练好的基础模型为起点，回答以下问题。

(a) 对多个学习率进行超参数扫描，报告最终损失；若优化器发散，则注明发散。

提交内容：

多个学习率对应的学习曲线。

对超参数搜索策略的解释。

一个在 TinyStories 上每 token 验证损失不高于 1.45 的模型。

低资源提示：在 CPU 或 Apple Silicon 上训练少量步骤

如果使用 cpu 或 mps，应把处理 token 总数降到 40,000,000，这仍足以生成较流畅的文本。目标验证损失也可从 1.45 放宽到 2.00。

使用课程组参考实现，在一台配备 36 GB 内存的 M4 Max 上，设置：

$\text{batch size} \times \text{总训练步数} \times \text{上下文长度} = 32 \times 5000 \times 256 = 40,960,000 \text{ token}。$

CPU 用时 1 小时 22 分钟，MPS 用时 36 分钟；第 5000 步验证损失达到 1.80。

额外提示：

若使用 N 个训练步骤，建议调整余弦学习率衰减调度，使其恰好在第 N 步完成衰减，也就是达到最小学习率。

使用 MPS 时，不要使用 TF32 kernel，也就是不要像 CUDA 设备那样设置：

```
torch.set_float32_matmul_precision('high')
```

课程组在 MPS 上测试了 TF32 kernel，PyTorch 版本为 2.9.0，发现后端有时会使用表面无报错但实际损坏的 kernel，导致训练不稳定。

可以用 `torch.compile` 对模型进行 JIT 编译，从而加快训练：

在 CPU 上：

```
model = torch.compile(model)
```

在 MPS 上，可以用以下方式在一定程度上优化反向传播：

```
model = torch.compile(model, backend="aot_eager")
```

截至 PyTorch 2.9.0，MPS 不支持使用 Inductor 编译。

(b) 一种经验观点认为，最佳学习率位于“稳定性边缘”。研究学习率开始发散的位置与最佳学习率之间的关系。

提交内容：

一组随学习率增大而变化的学习曲线，其中至少包含一次发散实验。

分析发散边界与收敛速度之间的关系。

接下来改变 batch size，观察训练会发生什么。较大的 batch 能让 GPU 进行更大的矩阵乘法，从而提高效率，但 batch 是否总是越大越好？通过实验寻找答案。

问题 (batch_size_experiment)：改变 batch size (1 B200 小时, 1 分)

把 batch size 从 1 一直改变到 GPU 内存上限。中间至少尝试若干取值，包括 64、128 等常见大小。

提交内容：

不同 batch size 对应的学习曲线。必要时应重新优化学习率。

用几句话讨论 batch size 对训练的影响及你的发现。

有了解码器之后，可以开始生成文本。生成质量至少应达到下面示例的水平。

示例 (ts_generate_example)：TinyStories 语言模型的生成样本

从前，有一个漂亮的女孩名叫莉莉。她喜欢吃口香糖，尤其喜欢那块黑色的大口香糖。

有一天，莉莉的妈妈让她帮忙做晚饭。莉莉非常兴奋！她喜欢帮助妈妈。妈妈为晚饭做了一大锅汤。莉莉非常开心，说：“谢谢你，妈妈！我爱你。”她帮妈妈把汤倒进一个大碗里。

晚饭后，莉莉的妈妈又做了一些美味的汤。莉莉非常喜欢！她说：“谢谢你，妈妈！这汤太好喝了！”妈妈笑着说：“你喜欢就好，莉莉。”她们完成了烹饪，又继续一起做饭。故事结束。

低资源提示：在 CPU 或 Apple Silicon 上生成文本

若使用只处理 40M token 的低资源配置，生成内容仍应有英语文本的样子，但流畅度不如上面的例子。

下面是课程组用一个在 40M token 上训练的 TinyStories 模型生成的原始样本。该样本本身有语义、代词和句法错误；以下译文尽量保留这种不自然感：

从前，有一个叫苏的小女孩。苏有一颗她非常喜欢的牙齿。那是他的最好的头。一天，苏出去散步，遇见了一只瓢虫！她们成了好朋友，一起在小路上玩。

“嘿，波莉！我们出去吧！”蒂姆说。苏看着天空，发现很难找到一种跳舞发亮的方法。她笑了，并同意帮助那个说话！“

当苏看着天空移动时，它是什么。她……

下面是准确的题目要求。

问题 (generate)：生成文本 (1 分)

使用解码器和训练好的检查点，报告模型生成的文本。为了获得流畅输出，可能需要调节温度、top-p 等解码参数。

提交内容：

至少 256 个 token 的文本输出；若提前生成第一个 `<|endoftext|>`，则输出到该 token 为止。

简短评价输出的流畅度。

至少列出两个会影响输出质量的因素。

7.3 消融实验与架构修改

理解 Transformer 的最佳方式，是亲自修改并观察模型行为。下面进行若干简单消融与修改。

消融 1：层归一化

人们常说层归一化对 Transformer 训练稳定性至关重要。但也许我们愿意冒险。移除每个 Transformer 块中的 RMSNorm，观察会发生什么。

问题 (layer_norm_ablation)：移除 RMSNorm 并训练 (0.5 B200 小时, 1 分)

从 Transformer 中移除全部 RMSNorm 并训练。在此前最佳学习率下会发生什么？降低学习率能否恢复稳定性？

提交内容：

移除 RMSNorm 后训练的学习曲线。

在新的最佳学习率下训练的学习曲线。

用几句话讨论 RMSNorm 的影响。

接下来研究另一个乍看似任意的层归一化选择。Pre-norm Transformer 块定义为：

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)) \quad (25)$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)) \quad (26)$$

它是原始 Transformer 架构为数不多的“共识性”改动之一。原始 Transformer 使用 post-norm：

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)) \quad (27)$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)) \quad (28)$$

现在恢复 post-norm，看看会发生什么。

问题 (pre_norm_ablation)：实现 post-norm 并训练 (0.5 B200 小时, 1 分)

把 pre-norm Transformer 修改为 post-norm，使用 post-norm 模型训练并观察结果。

提交内容：post-norm Transformer 的学习曲线，并与 pre-norm 对比。

由此可以看到，层归一化会显著影响 Transformer 行为，甚至层归一化所处位置也很重要。

消融 2：位置嵌入

接下来研究位置嵌入对模型性能的影响。具体而言，把使用 RoPE 的基础模型与完全不使用位置嵌入的模型 NoPE 进行比较。

理论上，decoder-only Transformer，也就是像本作业这样使用因果掩码的 Transformer，即使不显式提供位置嵌入，也可能推断相对或绝对位置信息 [Y.-H. H. Tsai 等, 2019; A. Kazemnejad 等, 2023]。现在通过实验比较 NoPE 与 RoPE。

问题 (no_pos_emb)：实现 NoPE (0.5 B200 小时, 1 分)

修改使用 RoPE 的 Transformer 实现，完全移除位置信息，观察结果。

提交内容： 比较 RoPE 与 NoPE 性能的学习曲线。

消融 3: SwiGLU 与 SiLU

接下来按照 N. Shazeer [20]，研究前馈网络中门控的重要性。比较 SwiGLU 前馈网络与使用 SiLU 激活但不含 GLU 的前馈网络：

$$\text{FFN_SiLU}(x) = W_2 \text{SiLU}(W_1 x) \quad (29)$$

回顾一下，SwiGLU 实现中把内部前馈层维度设为：

$$d_{\text{ff}} \approx (8/3) d_{\text{model}}$$

同时确保 $d_{\text{ff}} \bmod 64 = 0$ ，以使用 GPU tensor core。

在此消融基线中，FFN_SiLU 应改用：

$$d_{\text{ff}} = 4 \times d_{\text{model}}$$

这样可以近似匹配默认 SwiGLU 前馈网络的参数量，因为 SwiGLU 有三个权重矩阵，而 SiLU 基线只有两个。

问题 (swiglu_ablation)：SwiGLU 与 SiLU (0.5 B200 小时, 1 分)

提交内容：

在参数量大致匹配时，比较 SwiGLU 与 SiLU 前馈网络性能的学习曲线。

用几句话讨论你的发现。

低资源提示：GPU 资源有限的在线学习者应在 TinyStories 上测试修改

作业剩余部分将转向更大、噪声更多的网页数据集 OpenWebText，进行架构修改实验，并可选提交课程排行榜。

在 OpenWebText 上把语言模型训练到流畅需要很长时间，因此建议 GPU 资源有限的在线学习者继续在 TinyStories 上测试修改，并使用验证损失作为性能指标。

7.4 在 OpenWebText 上运行

现在转向一个更标准的、由网页抓取构建的预训练数据集。课程提供了 OpenWebText [A. Gokaslan 等, 2019] 的一个小型样本，保存为单个文本文件；访问方式见第 1 节。

下面给出一个 OpenWebText 样本。可以看到，这些文本比 TinyStories 更真实、更复杂、变化更多。建议浏览训练集，了解网页抓取语料的实际形态。

示例 (owt_example)：OpenWebText 样本

Baseball Prospectus 的技术总监 Harry Pavlidis 在聘请 Jonathan Judge 时冒了一次险。

Pavlidis 知道，正如 Alan Schwarz 在《The Numbers Game》中所写：“美国文化中，没有哪个角落像棒球运动员的表现那样，被如此精确地计数、如此热情地量化。”只需点几下鼠标，你就能查到 Noah Syndergaard 的快速球在飞向本垒板的途中每分钟旋转超过 2,100 次；Nelson Cruz 在 2016 年所有达到统计资格的击球手中，平均击球初速度最高；还可以找到无数其他细节，它们仿佛来自电子游

戏或科幻小说。不断上涨的数据海洋赋予了棒球文化中一种越来越重要的角色更多力量：分析型业余爱好者。

这种赋能也带来了更多审视，不仅审视测量结果，也审视结果背后的人和出版物。在 Baseball Prospectus 工作的 Pavlidis 非常了解量化结果不完美时会引发的反弹。他也知道，网站的接球指标需要重新设计，而完成这项工作需要一个训练有素的头脑，也就是能处理复杂统计建模问题的人。

“他让我们感到害怕。”——Harry Pavlidis

根据 Judge 的文章，以及二人在网站赞助的一次球场活动中的互动，Pavlidis 有一种直觉：Judge “懂这个”。……

注意： 本实验可能需要重新调节学习率、batch size 等超参数。

问题 (main_experiment)：在 OWT 上实验 (2 B200 小时, 2 分)

使用与 TinyStories 相同的模型架构和总训练迭代数，在 OpenWebText 上训练语言模型。模型表现如何？

提交内容：

OpenWebText 上的语言模型学习曲线。

描述其损失与 TinyStories 的差异，并说明应如何解释这些损失。

按照 TinyStories 输出的相同格式，给出 OpenWebText 语言模型生成的文本。

评价文本流畅度，并解释为什么在模型和计算预算相同的情况下，输出质量反而比 TinyStories 更差。

7.5 自定义修改与排行榜

恭喜你完成到这里，作业即将结束。现在尝试改进 Transformer 架构，并观察你的超参数和架构与班级其他同学相比表现如何。

排行榜规则

除以下限制外，没有其他限制。

运行时间： 提交程序在一张 B200 上最多运行 45 分钟。若使用 SLURM 或 Modal，可以在提交脚本中主动强制该时间限制。

数据： 只能使用课程提供的 OpenWebText 训练数据集。

除此之外，可以自由发挥。

若需要实现思路，可参考：

当前最先进的开源大语言模型家族，例如 Llama 3 [A. Grattafiori 等, 2024] 或 Qwen 2.5 [A. Yang 等, 2024]。

NanoGPT speedrun 仓库：github.com/KellerJordan/modded-nanogpt。社区成员会在其中发布许多用于“小规模语言模型预训练竞速”的有趣修改。

例如，一个可以追溯到原始 Transformer 论文的常见修改，是让输入嵌入与输出嵌入共享权重，见 A. Vaswani 等 [8] 第 3.4 节，以及 A. Chowdhery 等 [16] 第 2 节。若尝试权重共享，可能需要减小嵌入或 LM head 初始化的标准差。

正式进行完整的 45 分钟训练前，应先在 OpenWebText 的小型子集或 TinyStories 上测试这些修改。

需要提醒的是，排行榜上效果较好的某些修改，不一定能泛化到更大规模的预训练。课程将在缩放定律单元中进一步研究这一问题。

问题 (leaderboard)：排行榜 (10 B200 小时, 6 分)

在上述排行榜规则下训练模型，目标是在 0.75 B200 小时以内，尽可能降低语言模型验证损失。

提交内容：

最终记录的验证损失。

对应学习曲线，横轴必须清楚显示墙钟时间，且总时间小于 45 分钟。

对所做修改的描述。

排行榜提交应至少优于验证损失 5.0 的朴素基线。提交地址：

github.com/stanford-cs336/assignment1-basics-leaderboard

参考文献

- [1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?” 2023. (TinyStories: 语言模型可以小到什么程度，同时仍能说出连贯英语?)
- [2] A. Gokaslan, V. Cohen, E. Pavlick, and S. Tellex, “OpenWebText corpus.” 2019. (OpenWebText 语料库。)
- [3] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in Proc. of ACL, 2016. (使用子词单元进行稀有词神经机器翻译。)
- [4] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019. (使用字节级子词的神经机器翻译。)
- [5] P. Gage, “A new algorithm for data compression,” C Users Journal, vol. 12, no. 2, pp. 23-38, Feb. 1994. (一种新的数据压缩算法。)
- [6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019. (语言模型是无监督多任务学习器。)
- [7] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018. (通过生成式预训练改进语言理解。)
- [8] A. Vaswani et al., “Attention is All you Need,” in Proc. of NeurIPS, 2017. (Attention Is All You Need。)

- [9] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in Proc. of IWSWLT, 2019. (不再为 Transformer 流泪: 改进自注意力归一化。)
- [10] R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” in Proc. of ICML, 2020. (论 Transformer 架构中的层归一化。)
- [11] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016. (层归一化。)
- [12] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023. (LLaMA: 开放且高效的基础语言模型。)
- [13] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in Proc. of NeurIPS, 2019. (均方根层归一化。)
- [14] A. Grattafiori et al., “The Llama 3 Herd of Models.” Available: <https://arxiv.org/abs/2407.21783> (Llama 3 模型家族。)
- [15] A. Yang et al., “Qwen2.5 Technical Report,” arXiv:2412.15115, 2024. (Qwen2.5 技术报告。)
- [16] A. Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways.” 2022. (PaLM: 使用 Pathways 扩展语言建模。)
- [17] D. Hendrycks and K. Gimpel, “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units.” 2016. (使用高斯误差线性单元连接非线性函数与随机正则化器。)
- [18] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” Available: <https://arxiv.org/abs/1702.03118> (用于强化学习神经网络函数逼近的 Sigmoid 加权线性单元。)
- [19] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” Available: <https://arxiv.org/abs/1612.08083> (使用门控卷积网络进行语言建模。)
- [20] N. Shazeer, “GLU Variants Improve Transformer.” 2020. (GLU 变体改进 Transformer。)
- [21] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021. (RoFormer: 使用旋转位置嵌入增强 Transformer。)
- [22] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. of ICLR, 2015. (Adam: 一种随机优化方法。)
- [23] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in Proc. of ICLR, 2019. (解耦权重衰减正则化。)

- [24] T. B. Brown et al., “Language Models are Few-Shot Learners,” in Proc. of NeurIPS, 2020. (语言模型是少样本学习器。)
- [25] J. Kaplan et al., “Scaling Laws for Neural Language Models.” 2020. (神经语言模型的缩放定律。)
- [26] J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022. (训练计算最优的大语言模型。)
- [27] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in Proc. of ICLR, 2020. (神经文本退化的奇特案例。)
- [28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, and R. Salakhutdinov, “Transformer Dissection: An Unified Understanding for Transformer’ s Attention via the Lens of Kernel,” in EMNLP-IJCNLP, Hong Kong, China, Nov. 2019, pp. 4344-4353. doi: 10.18653/v1/D19-1443. (Transformer 解剖：从核方法视角统一理解 Transformer 注意力。)
- [29] A. Kazemnejad, I. Padhi, K. Natesan, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” NeurIPS 2023. Available: <https://openreview.net/forum?id=Drrl2gcjzl> (位置编码对 Transformer 长度泛化能力的影响。)